

Universitat de Girona
Escola Politècnica Superior

Grau en Enginyeria Informàtica

PROJECTE FINAL DE GRAU

**Terralink: En pobles o ciutats sempre
connectats. La tecnologia senzilla que
protegeix els nostres estimats**

Autor:
Pau Domènech Villahermosa

Tutors:
Dr. Adrià Julià i Juanola

MEMÒRIA

Convocatòria:
Setembre 2026

Departament :
Informàtica, Matemàtica Aplicada i Estadística

Projecte: Projecte Final de Grau
Document: Memòria
Títol: Terralink: En pobles o ciutats sempre connectats. La tecnologia senzilla que protegeix els nostres estimats
Autor: Pau Domènech Villahermosa
Data: Setembre 2026

Estudi:
Grau en Enginyeria Informàtica
Universitat de Girona

Supervisor 1:
Dr. Adrià Julià i Juanola
Universitat de Girona
Email: adria.julia@udg.edu
Web: [Link](#)

Índex

1	Introducció	1
1.1	Context del projecte	1
1.2	Motivació	1
1.3	Propòsit	1
1.4	Objectius	1
1.5	Abast del projecte	2
1.6	Estructura de la memòria	2
2	Estudi de viabilitat	4
2.1	Viabilitat tècnica	4
2.2	Viabilitat econòmica	5
2.3	Viabilitat legal i ètica	6
2.4	Riscos principals	7
2.5	Conclusió de viabilitat	7
3	Metodologia	8
3.1	Metodologia de treball	8
3.2	Organització i fases del desenvolupament	8
3.3	Eines utilitzades	9
3.4	Ús d'intel·ligència artificial generativa	9
4	Planificació	11
4.1	Pla de treball	11
4.2	Cronograma	11
4.3	Desviacions respecte a la planificació inicial	12
5	Marc de treball i conceptes previs	14
5.1	Marc de treball	14
5.2	Conceptes previs	14
5.2.1	LoRa i Meshtastic	14
5.2.2	Protocol ESC/POS	14
5.2.3	API Huawei HiLink	15
5.2.4	Automatització de WhatsApp Web	15
5.2.5	Transcripció de veu amb faster-whisper	15
6	Requisits del sistema	16
6.1	Actors	16
6.2	Requisits funcionals	16
6.3	Requisits no funcionals	17
7	Estudis i decisions	19
7.1	Elecció de Raspberry Pi	19

7.2	Elecció de LoRa/Meshtastic	19
7.3	Elecció del mòdem 4G/SMS	20
7.4	Elecció de pantalla, teclat físic, àudio i impressora	20
7.5	Decisions de programari	20
7.6	Alternatives descartades	21
8	Anàlisi i disseny del sistema	22
8.1	Actors i casos d'ús	22
8.2	Fitxes de casos d'ús	23
8.2.1	Fitxa 1. Enviar missatge d'emergència	25
8.2.2	Fitxa 2. Enviar missatge de text transcrit	26
8.2.3	Fitxa 3. Rebre missatge	27
8.2.4	Fitxa 4. Configurar WhatsApp i contactes	28
8.3	Model de dades	28
8.4	Diagrama de desplegament	29
8.5	Diagrama de components	30
9	Implementació i proves	32
9.1	Implementació del maquinari	32
9.1.1	Raspberry Pi 5 (unitat central)	32
9.1.2	Pantalla	32
9.1.3	Teclat Raspberry Pico (3 tecles + LEDs)	32
9.1.4	Mòdem 4G Huawei E3372	32
9.1.5	Micròfon	33
9.1.6	Impressora tèrmica ESC/POS	33
9.1.7	Mòdul LoRa i bateria	33
9.2	Implementació del programari	33
9.2.1	Estructura general del programari	34
9.2.2	Flux de funcionament del programari	34
9.3	Integració dels canals de comunicació	35
9.3.1	WhatsApp	35
9.3.2	SMS	35
9.3.3	LoRa	35
9.3.4	Impressió ESC/POS	35
9.4	Gestió de redundància	35
9.5	Proves unitàries	36
9.6	Proves d'integració	37
9.6.1	Proves de comunicació	37
9.6.2	Proves d'ús	37
10	Implantació i resultats	38
10.1	Procés d'implantació	38
10.2	Resultats	39
10.2.1	Assoliment dels requisits funcionals	39
10.2.2	Assoliment dels requisits no funcionals	39
10.3	Protecció de dades i normativa aplicable	40
11	Conclusions	41
11.1	Resum dels objectius i assoliment	41
11.2	Principals resultats obtinguts	42
11.3	Aportacions personals	42

11.4	Aprenentatge realitzat	42
12	Treball futur	43
A	Manuais d'instal·lació i d'usuari	45
A.1	Manual d'instal·lació (configuració de fàbrica)	45
A.1.1	Requisits previs	45
A.1.2	Instal·lació de dependències	45
A.1.3	Desplegament del codi	45
A.1.4	Configuració de l'arrencada automàtica	46
A.2	Manual d'usuari	46
A.2.1	Navegació bàsica	46
A.2.2	Enviar un missatge de veu	47
A.2.3	Rebre un missatge	47
A.2.4	Activar una emergència	47
A.2.5	Configuració (només per al Cuidador)	47
	Vincular el compte de WhatsApp	48
	Gestionar contactes	48
	Marcar contactes d'emergència	48
B	Prova amb <i>vibe coding</i>: prototip descartat	49
B.1	Objectiu de la prova	49
B.2	Problemes detectats	49
B.3	Conclusió	50
B.4	Codi del prototip	50
	Bibliografia	51

Capítol 1

Introducció

1.1 Context del projecte

Els sistemes de teleassistència actuals acostumen a dependre d'un únic canal de comunicació, com la xarxa mòbil o la línia fixa. Això en redueix la robustesa en situacions crítiques. En escenaris com talls elèctrics, incidències de xarxa o problemes puntuals de cobertura, aquesta dependència pot deixar les persones grans incomunicades just quan més necessiten poder contactar amb algú.

1.2 Motivació

Aquest projecte parteix d'una situació observada en l'entorn familiar de l'estudiant. D'una banda, al poble on resideix habitualment s'han donat, alguna vegada, talls de cobertura mòbil per vent i talls puntuals de subministrament elèctric. De l'altra, la persona gran que ha inspirat aquest projecte amb prou feines sap utilitzar un telèfon intel·ligent, i WhatsApp li resulta pràcticament inaccessible, tot i que podria comunicar-se amb normalitat si la interacció fos molt més senzilla (per exemple, gravant un missatge de veu en lloc d'haver d'escriure).

Aquesta situació posa de manifest dues necessitats: una interfície molt més simple que la d'un telèfon intel·ligent convencional, i un sistema que no depengui d'un sol canal de comunicació. D'aquí sorgeix la voluntat de dissenyar un dispositiu senzill que combini diversos canals i que no requereixi cap coneixement previ de WhatsApp ni de telèfons intel·ligents per fer-lo servir.

1.3 Propòsit

L'objectiu és desenvolupar un sistema de comunicació que permeti a una persona gran mantenir el contacte amb la seva família de manera senzilla i, sobretot, disposar d'una via garantida per enviar un avís en cas d'emergència, independentment del canal que estigui disponible en aquell moment. El sistema ha de ser fàcil d'utilitzar per a algú sense experiència amb dispositius electrònics i ha de continuar funcionant, dins del possible, quan algun dels mitjans de comunicació habituals falla.

1.4 Objectius

- Dissenyar un sistema de comunicació senzill i fiable per a persones grans.

- Integrar diversos canals de comunicació (WhatsApp, SMS i, en un futur, LoRa) que es puguin fer servir de manera complementària.
- Crear una interfície molt intuïtiva, accessible a una persona sense experiència prèvia amb la tecnologia.
- Garantir que un missatge d'emergència s'envii sempre que sigui possible, encara que un dels canals de comunicació falli.
- Demostrar la viabilitat tècnica i econòmica del sistema.

1.5 Abast del projecte

El projecte cobreix el disseny, la implementació i les proves d'un dispositiu de comunicació basat en una Raspberry Pi, integrant components de maquinari estàndard i desenvolupant el programari necessari per gestionar diversos canals de comunicació. S'inclou el disseny de la interfície d'usuari i la verificació del funcionament del sistema complet. El projecte no inclou el desplegament a llarg termini en un entorn real d'ús diari, ni la integració amb sistemes sanitaris o de teleassistència professional existents, aspectes que queden per al treball futur.

El codi complet del projecte es troba disponible a: <https://github.com/PauDomenech/tfg-terralink>

1.6 Estructura de la memòria

Aquesta memòria està organitzada en els capítols següents:

1. **Estudi de viabilitat:** analitza la viabilitat tècnica, econòmica i legal del projecte, i els riscos principals identificats.
2. **Metodologia:** descriu l'enfocament de desenvolupament seguit, les eines utilitzades i l'ús de la IA generativa.
3. **Planificació:** presenta el pla de treball, el cronograma i les desviacions respecte al pla inicial.
4. **Marc de treball i conceptes previs:** introdueix els conceptes tècnics necessaris per entendre la resta de la memòria.
5. **Requisits del sistema:** detalla els requisits funcionals i no funcionals que han guiat el disseny.
6. **Estudis i decisions:** justifica les eleccions tecnològiques preses i les alternatives descartades.
7. **Anàlisi i disseny del sistema:** presenta l'arquitectura del sistema, els casos d'ús i el model de dades.
8. **Implementació i proves:** descriu com s'ha implementat i provat el sistema.

9. **Implantació i resultats:** mostra els resultats obtinguts i el grau d'assoliment dels requisits.
10. **Conclusions:** avalua el projecte de manera crítica i recull l'aprenentatge adquirit.
11. **Treball futur:** proposa millores i ampliacions del sistema.

Capítol 2

Estudi de viabilitat

2.1 Viabilitat tècnica

Abans d'iniciar el desenvolupament, cal valorar si l'estudiant disposa (o pot adquirir en un temps raonable) de les competències tècniques necessàries per dur a terme el projecte. L'experiència prèvia de l'estudiant es limita a un contacte bàsic amb la Raspberry Pi (instal·lació del sistema operatiu i de programari) i a un petit programa desenvolupat amb Qt6 en el marc d'una assignatura de 4t de Configuració de Xarxes. La resta de tecnologies necessàries (gestió de mòdems 4G via API REST, integració amb WhatsApp mitjançant llibreries no oficials, comunicació LoRa/Meshtastic, transcripció de veu amb *faster-whisper* i impressió tèrmica ESC/POS) no formen part del bagatge previ de l'estudiant, però es tracta de tecnologies àmpliament documentades, amb comunitats actives i exemples d'ús disponibles, cosa que en fa previsible un aprenentatge autònom dins la durada prevista del TFG.

Es considera, doncs, que el projecte és tècnicament viable: la corba d'aprenentatge necessària és assumible per a un estudiant de grau amb els coneixements base de programació i sistemes que aporta la titulació, tot i la complexitat tecnològica global del sistema.

S'identifica, però, un risc tècnic rellevant a valorar abans de començar: la integració d'un mòdul LoRa directament als pins GPIO de la Raspberry Pi 5. Es tracta d'una placa relativament recent, i no tots els mòduls LoRa disponibles al mercat tenen la seva compatibilitat amb GPIO de la Raspberry Pi 5 verificada per la comunitat, més enllà de la compatibilitat teòrica anunciada pel fabricant. Això podria comportar problemes de reconeixement per part del sistema operatiu difícils de preveure sense fer-ne primer una prova pràctica, motiu pel qual es considera necessari reservar un marge de contingència per a aquesta tasca (Capítol 4) i tenir prevista una alternativa, com un mòdul LoRa en format USB, per si la integració via GPIO no fos viable.

Cal remarcar també que la Raspberry Pi 5 disposa d'un nombre limitat de ports USB, que es preveu que quedin ocupats pel mòdem 4G, el teclat, el micròfon i la impressora. Si finalment calgués recórrer a un mòdul LoRa en format USB, caldria preveure també l'adquisició d'un *hub* USB alimentat (per exemple, un hub PCIe a USB 3.2 Gen1 per a Raspberry Pi 5) per no quedar-se sense ports lliures.

2.2 Viabilitat econòmica

El pressupost del projecte s'ha mantingut deliberadament baix, aprofitant components de segona mà, ofertes puntuals i una donació de maquinari. La Taula 2.1 detalla el pressupost complet del dispositiu final: les hores de treball (a un preu fictici de mercat de 14€/h, equivalent al salari brut per hora d'un enginyer júnior) i el cost dels components de maquinari.

Pressupost TerraLink			
Concepte	Quantitat	Preu unitari	Total
Recerca de components i compatibilitat	60 h	14,00 €/h	840,00 €
Programació (Qt6/C++, Node.js, Python)	250 h	14,00 €/h	3.500,00 €
Muntatge i configuració de components	75 h	14,00 €/h	1.050,00 €
Raspberry Pi 5 (4GB, donada per Raspberry Pi)	1	—	0,00 €
Mòdem 4G Huawei E3372 (segona mà)	1	30,00 €	30,00 €
Mòduls LoRa	2	25,00 €	50,00 €
Impressora tèrmica ESC/POS	1	20,00 €	20,00 €
Pantalla	1	20,00 €	20,00 €
Teclat	1	20,00 €	20,00 €
Altaveu	1	5,00 €	5,00 €
Micròfon	1	3,00 €	3,00 €
Targeta SD (donada per Raspberry Pi)	1	—	0,00 €
Font d'alimentació 27W USB-C (donada per Raspberry Pi)	1	—	0,00 €
Cable Micro HDMI a HDMI (donat per Raspberry Pi)	2	—	0,00 €
Total (IVA inclòs)			5.538,00 €

TAULA 2.1: Pressupost estimat del projecte TerraLink

Cal destacar que el dispositiu final funciona amb una Raspberry Pi 5 de 4GB que l'empresa Raspberry Pi va proporcionar de forma gratuïta a l'estudiant. Aquesta donació incloïa un kit complet valorat en 168,99\$ (aproximadament 155€ al canvi), format per: Raspberry Pi 5 (85\$), font d'alimentació 27W USB-C (12\$), dos cables Micro HDMI a HDMI (5\$ cadascun, 10\$ en total), teclat (17\$), ratolí (8\$), targeta SD (2\$), carcassa amb ventilador (10\$) i manual/llibre (24,99\$). Aquest import es reflecteix a la Taula 2.1 amb cost 0€, per tractar-se d'una donació.

Val a dir que, abans de rebre aquesta donació, es va comprar inicialment una Raspberry Pi 5 de 8GB per 90€ per començar a explorar el projecte. Un cop rebut el kit gratuït, es va optar per continuar el desenvolupament amb la Raspberry Pi 5 de 4GB donada, ja que era suficient per als requisits del sistema. Per aquest motiu, els 90€ de la Raspberry Pi 5 de 8GB no s'inclouen al pressupost del dispositiu final: no forma part del producte acabat.

El total de 385 hores de dedicació equival a la càrrega lectiva dels 15 ECTS assignats al TFG. El cost total de **5.538 €** té únicament finalitat orientativa per justificar la viabilitat econòmica acadèmica del projecte, ja que en un context real d'estudiant no hi ha cap retribució econòmica associada.

Quant al treball futur (Secció 2.5 i Capítol 12), es preveu un cost addicional aproximat de 65-105€ per incorporar un mòdul LoRa USB (20-30€), un hub USB alimentat (15-25€) i un sistema de bateria/UPS (30-50€). Aquests elements encara estan pendents d'adquisició per motius pressupostaris.

Cal remarcar que el pressupost i les 385 hores anteriors corresponen a un **prototip** que implementa només una part de la visió inicial del producte (sense LoRa ni bateria pròpia, amb un sol canal de *fallback* per als missatges normals). Convertir TerraLink en un producte comercialitzable i complet requeriria una inversió d'hores molt superior a la d'aquest TFG: integració real de LoRa, disseny industrial de la carcassa, un sistema d'alimentació autònoma, proves de camp amb usuaris reals, certificacions (marcatge CE, normativa de telecomunicacions) i un procés de manteniment i actualització del programari. La viabilitat econòmica descrita en aquesta secció s'ha de llegir, doncs, com la d'un prototip funcional dins l'abast d'un TFG, no com la d'un producte final llest per al mercat.

2.3 Viabilitat legal i ètica

TerraLink emmagatzema localment, a la pròpia Raspberry Pi, dades de caràcter personal com els contactes de l'usuari (nom i número de telèfon). Els missatges rebuts, en canvi, no es persisteixen a disc: només es mostren en pantalla i, opcionalment, s'imprimeixen en paper mitjançant la impressora tèrmica. En cap moment aquestes dades surten del dispositiu cap a un servidor extern o una empresa tercera. Això redueix significativament les obligacions derivades del Reglament General de Protecció de Dades (RGPD) [1] i la Llei Orgànica de Protecció de Dades (LOPD) [2], ja que no hi ha cap responsable de tractament extern que reculli o processi aquesta informació. Cal dir, però, que aquest anàlisi és una aproximació orientativa pròpia d'un TFG i no substitueix un assessorament legal professional, recomanable en cas de comercialitzar el producte.

Pel que fa a la comunicació amb WhatsApp, aquesta es realitza mitjançant un procés Node.js que es comunica amb l'aplicació principal via entrada/sortida estàndard (*stdin/stdout*). Això no exposa cap port de xarxa ni API accessible externament, cosa que dificulta la intercepció de les dades per tercers.

Quant a la Llei de Serveis de la Societat de la Informació i Comerç Electrònic (LSSICE) [3], aquesta normativa regula principalment els serveis prestats per internet (comerç electrònic, allotjament web, comunicacions comercials, etc.), no el funcionament intern d'un dispositiu de maquinari. En la versió actual del projecte, desenvolupada com a TFG i sense comercialització, la LSSICE no és aplicable. Cal preveure, però, que en un futur escenari de comercialització del producte, instal·lat per un tècnic o per un familiar de l'usuari final seguint un manual de configuració, caldria valorar altres normatives més rellevants que la LSSICE, com el marcatge CE per a productes electrònics o la normativa de protecció de consumidors. Aquests aspectes queden fora de l'abast d'aquest TFG. Igualment, si en el futur es desplegués un mecanisme d'actualitzacions automàtiques del programari (contemplat com a treball futur al Capítol 12), caldria valorar-ne els aspectes de seguretat i la normativa aplicable en el moment de la seva implementació.

Finalment, pel que fa a l'ús d'intel·ligència artificial, el sistema utilitza *faster-whisper* per a la transcripció de veu a text. Es tracta d'un model executat íntegrament de forma local a la Raspberry Pi, sense enviar l'àudio de l'usuari a cap servidor extern ni servei de tercers. Això reforça la privacitat de l'usuari i simplifica les consideracions ètiques associades a l'ús de la IA en aquest projecte.

2.4 Riscos principals

S'han identificat els següents riscos principals durant el desenvolupament del projecte:

- **Risc regulador amb LoRa:** cal tenir clar que tot el subsistema LoRa/Meshtastic és, a tots els efectes, un sistema de telecomunicacions per ràdio, i com a tal està subjecte a la normativa de l'espectre radioelèctric. El risc principal no es limita a la potència de transmissió: els mòduls LoRa es poden configurar per programari a freqüències fora de la banda ISM autoritzada (868 MHz a Europa), cosa que suposaria una infracció encara que la potència emesa fos correcta. Com que finalment no es va aconseguir posar en marxa cap dels dos mòduls adquirits, aquest risc no s'ha arribat a materialitzar en aquesta versió del projecte. Caldrà validar explícitament tant la freqüència com la potència de configuració quan s'implementi LoRa en el treball futur.
- **Ús d'una llibreria no oficial de WhatsApp:** la integració amb WhatsApp es realitza mitjançant una llibreria de JavaScript no oficial que emula un client web. Aquest tipus de llibreries comporten el risc que Meta pugui bloquejar el número de telèfon si detecta un patró d'ús propi d'un compte de tipus *bot* o empresarial (que normalment requereix un pagament a través de l'API oficial de WhatsApp Business). En un ús convencional, com el previst per a un usuari particular (per exemple, la persona gran per a qui es dissenya el sistema), no s'espera que es produeixi aquest bloqueig: el volum i el patró de missatges és equiparable al d'un usuari humà normal.
- **Absència de sistema de bateria:** actualment el dispositiu no disposa de bateria pròpia. Això suposa un risc rellevant, tenint en compte que un dels objectius principals del projecte és garantir la comunicació en situacions d'emergència, incloent talls de subministrament elèctric. Aquest risc es documenta com a limitació actual i es proposa com a prioritat en el treball futur (Capítol 12).

Val la pena remarcar que la dependència d'un sol dispositiu de maquinari (tot el sistema recau sobre una única Raspberry Pi, sense redundància física) no s'ha de llegir únicament com un risc, sinó també com un avantatge des del punt de vista de la privacitat i del model de producte: en no dependre de cap servidor ni servei al núvol, les dades de l'usuari no surten mai del dispositiu, cosa que elimina el risc de filtració de dades a tercers.

2.5 Conclusió de viabilitat

Malgrat les limitacions detectades (especialment la no integració de LoRa i l'absència de bateria), el projecte es considera **viable** tant a nivell tècnic com econòmic. El baix cost del dispositiu final (uns 148€ en components de maquinari no donats, gràcies també a la donació de la Raspberry Pi) i la corba d'aprenentatge assumible per un estudiant de grau confirmen que TerraLink és un projecte realista dins l'abast d'un Treball Fi de Grau. Els riscos identificats són coneguts i estan documentats. No comprometen la viabilitat global del sistema, sinó que marquen clarament les línies de treball futur necessàries per completar la visió inicial del projecte.

Capítol 3

Metodologia

3.1 Metodologia de treball

El desenvolupament de TerraLink ha seguit un enfocament proper a les metodologies àgils, tot i que de forma informal i adaptada a les característiques d'un projecte individual de Treball Fi de Grau. En lloc de definir sprints tancats amb una durada fixa, el projecte ha avançat de manera incremental i contínua. La dedicació ha estat setmanal, amb l'excepció dels períodes d'exàmens, on es va reduir considerablement.

Aquest enfocament ha permès validar cada part del sistema abans de passar a la següent, evitant haver de replantejar grans blocs de treball ja fets. Per exemple, no es va començar a implementar cap classe en C++ fins que el disseny (diagrames de classes) de l'apartat corresponent no estava prou madur. Això ha reduït els canvis estructurals durant la fase d'implementació.

3.2 Organització i fases del desenvolupament

El desenvolupament del projecte s'ha organitzat en quatre grans fases, executades de manera seqüencial però amb certa superposició entre elles:

1. **Recerca de components i estudi de viabilitat:** identificació i adquisició del maquinari necessari (Raspberry Pi, mòdem 4G, mòduls LoRa, impressora tèrmica, etc.). Inclou les proves de compatibilitat que van portar a descartar temporalment la integració de LoRa (veure Capítol 2).
2. **Disseny:** un cop identificats els components disponibles, es van elaborar els diagrames de classes, el diagrama de desplegament i la resta de documents que defineixen l'arquitectura del sistema (Capítol 8).
3. **Programació:** implementació del codi en Qt6/C++ seguint el disseny establert prèviament. Inclou la integració amb els diferents adaptadors de maquinari i serveis externs (Capítol 9).
4. **Proves:** validació del funcionament del sistema complet, tant a nivell d'unitats individuals com d'integració entre components.

Pel que fa al control de versions, cal remarcar com a limitació metodològica que el

projecte **no ha utilitzat cap sistema de control de versions** (com Git). Tot el desenvolupament s'ha dut a terme directament sobre el sistema de fitxers de la Raspberry Pi, sense mantenir un historial de canvis ni poder revertir fàcilment a versions anteriors del codi. Aquesta decisió no ha estat intencionada, sinó fruit de la manca d'experiència prèvia amb aquestes eines. De cara a futurs projectes, s'identifica com una millora clara a incorporar des del primer dia de desenvolupament.

3.3 Eines utilitzades

Per al desenvolupament de TerraLink s'han utilitzat les eines i tecnologies següents:

- **Llenguatges i frameworks:** Qt6/C++ per a l'aplicació principal, Node.js (amb la llibreria *whatsapp-web.js*) per a la integració amb WhatsApp, i Python (amb *faster-whisper*) per a la transcripció de veu.
- **Entorn de desenvolupament:** Qt Creator [4] per a la implementació i depuració del codi C++, i l'editor de codi integrat de Raspberry Pi OS per a scripts puntuals de Node.js i Python.
- **Documentació del codi:** Doxygen [5] per a la documentació tècnica de les classes C++.
- **Diagrames:** draw.io [6] per a l'elaboració dels diagrames de classes, de components i de desplegament.
- **Redacció de la memòria:** L^AT_EX, seguint la plantilla oficial de TFG de la UdG.
- **Control de versions:** no se n'ha utilitzat cap, tal com s'ha comentat a la Secció 3.2.

3.4 Ús d'intel·ligència artificial generativa

Durant l'elaboració d'aquest TFG s'ha utilitzat Claude (Anthropic) com a eina d'intel·ligència artificial generativa, tant en la fase d'implementació del codi com en la redacció de la memòria. A continuació es detalla l'abast d'aquest ús:

- **Suport en la implementació i depuració del codi:** Claude s'ha utilitzat principalment com a eina d'ajuda per depurar errors de compilació i d'execució, i per rebre suggeriments sobre bones pràctiques de disseny orientat a objectes. S'estima que aproximadament un **10-20% del codi final** ha estat generat o modificat amb l'assistència d'aquesta eina. La resta ha estat escrit i dissenyat directament per l'estudiant.

Aquest ús puntual no s'ha de confondre amb una prova diferent i anterior, feta només a l'inici de la fase d'implementació: un petit **prototip generat íntegrament mitjançant *vibe coding*** (Secció 4.3 i Apèndix B), sense disseny ni revisió manual del codi. Aquest prototip no té relació amb el codi final: es va descartar en la seva totalitat i no en forma part en cap grau.

- **Suport en la redacció de la memòria:** Claude també s'ha utilitzat per millorar la redacció i l'expressió escrita d'aquest document, ajudant a estructurar el contingut i a expressar en un llenguatge més formal les idees i decisions preses

per l'estudiant al llarg del projecte. En cap cas s'ha delegat a la IA la presa de decisions tècniques ni de contingut. Totes les dades, xifres, decisions de disseny i conclusions reflectides en aquesta memòria són fruit del treball i el criteri de l'estudiant.

No s'ha utilitzat cap altra eina d'intel·ligència artificial generativa (com ChatGPT, GitHub Copilot o Gemini) durant el desenvolupament d'aquest projecte.

Capítol 4

Planificació

4.1 Pla de treball

El projecte es va iniciar el desembre de 2025 i finalitza al juliol de 2026, coincidint amb l'últim termini de dipòsit del curs. Aquesta planificació suposa una durada total d'aproximadament 8 mesos, amb una dedicació total de 385 hores (equivalent als 15 ECTS del TFG), distribuïdes en cinc paquets de treball (PT):

- **PT1 – Recerca de components i estudi de viabilitat** (60h): cerca i adquisició del maquinari (Raspberry Pi, mòdem 4G, mòduls LoRa, impressora tèrmica, etc.), comparant preus i ofertes a AliExpress i altres proveïdors, i contacte amb Raspberry Pi per a l'obtenció del kit donat. Documentat al Capítol 2.
- **PT2 – Comprovació de funcionament i compatibilitat dels components** (35h): proves individuals de cada component per determinar-ne la compatibilitat real amb la Raspberry Pi 5. Inclou les proves fallides d'integració del mòdul LoRa. Documentat al Capítol 2.
- **PT3 – Disseny** (40h): elaboració dels diagrames de classes, del diagrama de desplegament i de la resta de decisions d'arquitectura del sistema. Documentat al Capítol 8.
- **PT4 – Implementació** (210h): programació de l'aplicació en Qt6/C++, incloent la integració amb WhatsApp (Node.js), el mòdem 4G (API Huawei HiLink), la impressora ESC/POS i la transcripció de veu (Python/faster-whisper). Documentat al Capítol 9.
- **PT5 – Muntatge final, proves i redacció de la memòria** (40h): muntatge físic definitiu del dispositiu, proves d'integració del sistema complet i redacció d'aquesta memòria. Documentat als Capítols 9, 10 i el present capítol.

4.2 Cronograma

La Taula 4.1 mostra la distribució temporal dels cinc paquets de treball al llarg dels 8 mesos de durada del projecte.

Paquet	Des	Gen	Feb	Mar	Abr	Mai	Jun	Jul
PT1 – Recerca i viabilitat	X	X						
PT2 – Comprovació components		X	X					
PT3 – Disseny			X	X				
PT4 – Implementació				X	X	X	X	
PT5 – Muntatge final, proves i memòria							X	X

TAULA 4.1: Cronograma del projecte per paquets de treball

Com es pot observar, hi ha una certa superposició entre paquets consecutius (per exemple, entre PT2 i PT3, o entre PT4 i PT5). En la pràctica, algunes tasques de disseny es van avançar mentre encara es validaven components, i la redacció de la memòria s'ha anat fent de manera progressiva en paral·lel a les últimes fases d'implementació i proves.

4.3 Desviacions respecte a la planificació inicial

Durant el desenvolupament del projecte s'han produït algunes desviacions respecte a la planificació inicial:

- **Incompatibilitat del mòdul LoRa:** tal com es detalla al Capítol 2, es van invertir hores addicionals en la compra i les proves de dos mòduls LoRa diferents que finalment no es van poder integrar amb la Raspberry Pi 5. Aquest temps, inicialment previst per a la integració completa de LoRa, es va haver de reorientar cap al descart justificat d'aquesta funcionalitat i la seva reformulació com a treball futur.
- **Prova inicial amb *vibe coding*:** a l'inici de la fase d'implementació, es va fer una primera prova ràpida de codi generat de forma majoritàriament automàtica amb eines d'intel·ligència artificial. L'objectiu era validar que era tècnicament possible comunicar-se amb cada component de maquinari (mòdem, impressora, etc.). El codi resultant d'aquesta prova no tenia prou qualitat ni estructura per formar part del projecte final, i es va descartar íntegrament. Sí que va servir, però, per identificar ràpidament quines llibreries i crides eren necessàries per a cada component, i per confirmar la importància de dissenyar i implementar el codi final de manera manual i acurada, en lloc de dependre de la IA per a aquesta tasca.
- **Alentiment durant els períodes d'exàmens:** la dedicació setmanal es va reduir significativament durant els períodes d'exàmens de 3r curs, especialment per assignatures de dificultat elevada com Paradigmes i Llenguatges de Programació. Aquests períodes van suposar diverses setmanes de treball pràcticament aturat, cosa que va allargar lleugerament algunes de les fases previstes.
- **Refactorització a mitja implementació:** durant la fase d'implementació es va detectar que la classe inicial `AdaptadorModem4G` gestionava alhora la comunicació amb WhatsApp i amb SMS, cosa que en dificultava el manteniment. Es va decidir dividir-la en dues classes independents, `AdaptadorWhatsApp` i `AdaptadorModem4G` (aquesta última limitada exclusivament a SMS), tal com es detalla al Capítol 9. Aquesta refactorització no era prevista inicialment, però

es considera una desviació menor que ha millorat la qualitat de l'arquitectura final del sistema.

Malgrat aquestes desviacions, el projecte s'ha pogut tancar dins del termini previst de 8 mesos, gràcies a la flexibilitat de l'enfocament àgil adoptat i a la reorientació ràpida de les tasques afectades. Cal ser precís, però, sobre l'abast d'aquest tancament: s'han completat les parts més importants i crítiques del sistema (enviament i recepció per WhatsApp i SMS, transcripció de veu, impressió i el protocol d'emergència multicanal), però no la totalitat de la visió inicial del projecte. Funcionalitats previstes en el disseny original, com la integració de LoRa, l'alimentació per bateria o el *fallback* automàtic per als missatges normals, queden pendents com a treball futur (Capítol 12). El resultat d'aquest TFG és, doncs, un prototip funcional i sòlid, no el producte complet plantejat als objectius inicials.

Capítol 5

Marc de treball i conceptes previs

5.1 Marc de treball

TerraLink és un projecte d'iniciativa pròpia de l'estudiant, tutoritzat pel Dr. Adrià Julià i Juanola de la Universitat de Girona. No es desenvolupa en el marc d'una empresa, institució o grup de recerca, sinó com a proposta personal sorgida d'una motivació concreta que ja s'ha detallat al Capítol 1: la voluntat de facilitar la comunicació d'una persona gran de l'entorn familiar de l'estudiant amb les seves persones estimades, mitjançant un dispositiu senzill que no depengui de l'ús d'un telèfon intel·ligent convencional.

El projecte no parteix de cap sistema o codi previ existent. Tot el maquinari, el disseny i el codi s'han desenvolupat des de zero específicament per a aquest TFG. Per aquest motiu, aquest capítol es limita a introduir els conceptes tècnics necessaris per entendre la resta de la memòria, sense haver de descriure cap entorn corporatiu o de recerca previ.

5.2 Conceptes previs

A continuació es descriuen breument les tecnologies i protocols més específics utilitzats en el projecte, necessaris per entendre correctament els capítols de disseny i implementació.

5.2.1 LoRa i Meshtastic

LoRa (*Long Range*) és una tecnologia de comunicació sense fils de llarg abast i baix consum energètic, dissenyada per transmetre petites quantitats de dades a distàncies de diversos quilòmetres sense necessitat de cobertura mòbil ni connexió a internet. Meshtastic [7, 8] és un projecte de programari lliure que utilitza mòduls de ràdio LoRa per crear xarxes en malla (*mesh*) entre dispositius. Permet l'intercanvi de missatges de text entre nodes sense necessitat d'infraestructura de xarxa centralitzada. En el context de TerraLink, aquesta tecnologia es contemplava com a canal de comunicació alternatiu per a zones sense cobertura mòbil, tot i que finalment no s'ha pogut integrar en aquesta versió del projecte (veure Capítol 2).

5.2.2 Protocol ESC/POS

ESC/POS [9] és un llenguatge de comandes desenvolupat originalment per Epson, àmpliament adoptat com a estàndard *de facto* per a impressores tèrmiques de tiquets.

Consisteix en seqüències de bytes especials (que comencen normalment pel caràcter ESC) que la impressora interpreta com a instruccions en lloc d'imprimir-les com a text: per exemple, per activar el mode de negreta, tallar el paper o canviar la codificació de caràcters. TerraLink utilitza aquest protocol per comunicar-se directament amb la impressora tèrmica connectada per USB, escrivint aquestes seqüències de bytes al dispositiu del sistema corresponent (`/dev/usb/lp0`).

5.2.3 API Huawei HiLink

HiLink és el mode de funcionament dels mòdems USB 4G de Huawei, com el model E3372 [10] utilitzat en aquest projecte. Exposa una interfície de gestió web accessible per HTTP a una adreça IP local (típicament `192.168.8.1`). Aquesta interfície ofereix una API REST no oficial, basada en peticions XML, que permet enviar i rebre SMS, consultar l'estat de la connexió i gestionar el dispositiu de manera programàtica, sense necessitat d'utilitzar cap client gràfic. TerraLink es comunica amb aquesta API per enviar i rebre missatges SMS de manera automatitzada.

5.2.4 Automatització de WhatsApp Web

WhatsApp no ofereix una API pública gratuïta per a l'enviament automatitzat de missatges des de comptes personals. Per solucionar-ho, existeixen llibreries de codi obert, com *whatsapp-web.js* [11], utilitzada en aquest projecte, que emulen el funcionament d'un navegador connectat a WhatsApp Web. Això permet enviar i rebre missatges de manera programàtica des d'un script de Node.js. Aquestes llibreries no estan avalades oficialment per WhatsApp/Meta, cosa que comporta certs riscos que ja s'han discutit a l'apartat de riscos del Capítol 2.

5.2.5 Transcripció de veu amb faster-whisper

faster-whisper [12] és una implementació optimitzada del model de reconeixement de veu Whisper (desenvolupat per OpenAI), dissenyada per executar-se de manera eficient en maquinari amb recursos limitats, com és el cas de la Raspberry Pi. A diferència d'altres solucions de transcripció que requereixen enviar l'àudio a un servidor extern, *faster-whisper* s'executa íntegrament de forma local. TerraLink aprofita això per transcriure els missatges de veu de l'usuari a text sense comprometre'n la privacitat.

Capítol 6

Requisits del sistema

6.1 Actors

El sistema TerraLink identifica dos actors principals:

- **Persona Dependent:** persona gran que utilitza directament el dispositiu per enviar i rebre missatges de veu, gestionar les seves emergències i mantenir el contacte amb els seus familiars. És l'actor principal per al qual s'ha dissenyat tota la interfície d'usuari, pensada per requerir el mínim d'interacció possible (navegació amb poques tecles físiques).
- **Cuidador:** familiar o persona amb coneixements tècnics bàsics responsable de la configuració inicial del sistema: vincular el compte de WhatsApp mitjançant el codi QR, importar o gestionar els contactes, configurar els canals de comunicació disponibles i marcar quins contactes han de rebre els missatges d'emergència. Aquest actor també pot rebre missatges enviats per la Persona Dependent i enviar-n'hi de nous. Accedeix a la configuració del sistema mitjançant la combinació secreta de tecles descrita al Capítol 8.

Els contactes de la Persona Dependent que no actuen com a Cuidador (altres familiars o persones properes) no interactuen directament amb el dispositiu TerraLink. Reben i responen els missatges des dels seus propis dispositius (WhatsApp o SMS), per la qual cosa no es consideren actors del sistema.

6.2 Requisits funcionals

La Taula 6.1 resumeix els requisits funcionals identificats per al sistema, agrupats per actor i amb la seva prioritat associada.

ID	Requisit	Prioritat
RF1	Enviament de missatges de veu transcrits a text	Alta
RF2	Recepció i impressió de missatges entrants	Alta
RF3	Activació del protocol d'emergència	Alta
RF4	Gestió de contactes	Mitjana
RF5	Importació de contactes des de WhatsApp	Mitjana
RF6	Vinculació del compte de WhatsApp mitjançant codi QR	Alta
RF7	Enviament i recepció de missatges per SMS	Mitjana

TAULA 6.1: Requisits funcionals del sistema

RF1 – Enviament de missatges de veu transcrits a text. La Persona Dependent ha de poder gravar un missatge de veu, que el sistema transcriu automàticament a text i envia al Cuidador (o a un altre contacte seleccionat) pel canal disponible, prioritàriament WhatsApp.

RF2 – Recepció i impressió de missatges entrants. El sistema ha de mostrar en pantalla els missatges rebuts (WhatsApp o SMS) i permetre imprimir-los en paper mitjançant la impressora tèrmica, per facilitar-ne la lectura per part de la Persona Dependent.

RF3 – Activació del protocol d'emergència. La Persona Dependent ha de poder activar, amb una única acció, l'enviament d'un missatge d'emergència a tots els contactes marcats com a tals, pel canal de comunicació disponible.

RF4 – Gestió de contactes. El Cuidador ha de poder afegir, modificar i eliminar contactes, així com marcar-los com a contactes d'emergència.

RF5 – Importació de contactes des de WhatsApp. El sistema ha de permetre al Cuidador importar automàticament els contactes ja existents al compte de WhatsApp vinculat, filtrant aquells que no corresponguin a números de telèfon vàlids.

RF6 – Vinculació del compte de WhatsApp mitjançant codi QR. El Cuidador ha de poder vincular el dispositiu a un compte de WhatsApp mostrant el codi QR corresponent, seguint el mecanisme estàndard de WhatsApp Web.

RF7 – Enviament i recepció de missatges per SMS. Quan el canal de WhatsApp no estigui disponible, el sistema ha de poder enviar i rebre missatges mitjançant SMS a través del mòdem 4G.

6.3 Requisits no funcionals

A banda dels requisits funcionals descrits a la secció anterior, el sistema ha de complir una sèrie de requisits no funcionals que en condicionen el disseny i la implementació. La Taula 6.2 els resumeix, i tot seguit es detalla cadascun.

ID	Requisit	Prioritat
RNF1	Usabilitat per a persones grans	Alta
RNF2	Fiabilitat i redundància de canals	Alta
RNF3	Privacitat de les dades	Alta
RNF4	Disponibilitat contínua	Alta
RNF5	Facilitat d'expansió	Mitjana

TAULA 6.2: Requisits no funcionals del sistema

RNF1 – Usabilitat per a persones grans. El sistema es destina a un usuari final gran, sense experiència prèvia amb dispositius electrònics complexos (Capítol 1). Per aquest motiu, tota la interacció s'ha de poder fer amb un nombre reduït de tecles físiques, evitant menús amb massa nivells de profunditat o accions que requereixin precisió motora fina.

RNF2 – Fiabilitat i redundància de canals. El sistema té com a finalitat principal mantenir la comunicació de l'usuari, especialment en situacions d'emergència. Per això, la fallada d'un únic canal de comunicació no pot deixar l'usuari incomunicat: si el canal principal (WhatsApp) no està disponible, el sistema ha de poder recórrer a un canal alternatiu (SMS).

RNF3 – Privacitat de les dades. Les dades personals de l'usuari (contactes) i el contingut dels missatges gestionats pel sistema no han de sortir del dispositiu ni ser accessibles per tercers, més enllà del que és estrictament necessari per al funcionament dels canals de comunicació utilitzats.

RNF4 – Disponibilitat contínua. El sistema ha d'estar operatiu de manera constant, sense requerir intervencions manuals de reinici o manteniment per part de l'usuari final per continuar funcionant correctament al llarg del dia.

RNF5 – Facilitat d'expansió. L'arquitectura del sistema ha de permetre afegir nous canals de comunicació o noves funcionalitats en el futur (per exemple, LoRa o sensors addicionals, tal com es detalla al Capítol 12) sense haver de modificar el codi ja existent de manera substancial.

Capítol 7

Estudis i decisions

7.1 Elecció de Raspberry Pi

Es va escollir la Raspberry Pi 5 com a maquinari central del sistema per dos motius principals. En primer lloc, per la seva fiabilitat contrastada: es tracta d'una placa àmpliament utilitzada tant en l'àmbit educatiu com en projectes de producció, amb un suport de comunitat i documentació molt extens. En segon lloc, perquè el sistema necessita executar simultàniament diversos processos amb una certa càrrega computacional: l'aplicació Qt6/C++ principal, el procés Node.js del pont de WhatsApp i el motor de transcripció de veu *faster-whisper*. Tot això sense que se'n ressenti la fluïdesa.

Alternatives com un microcontrolador tipus Arduino [13] es van descartar directament, ja que no disposen de sistema operatiu ni de la potència de càlcul necessària per executar aquests processos. Un mini-PC convencional oferiria prestacions similars o superiors, però amb un cost i unes dimensions físiques molt més elevades, allunyant-se de l'objectiu d'un dispositiu senzill i compacte. Altres plaques d'ordinador d'una sola placa (SBC), com les Orange Pi, es van descartar per tenir una comunitat de suport i una documentació molt més limitades que les de Raspberry Pi.

7.2 Elecció de LoRa/Meshtastic

La tecnologia LoRa es va escollir com a canal de comunicació previst per a situacions sense cobertura de xarxa mòbil, principalment per dos motius. D'una banda, el tutor del projecte tenia experiència prèvia amb aquesta tecnologia, cosa que en facilitava l'orientació durant el desenvolupament. De l'altra, LoRa permet establir comunicació independentment de la cobertura 3G/4G, un aspecte especialment rellevant per a l'objectiu del projecte de garantir la comunicació de l'usuari en qualsevol circumstància.

Es van descartar alternatives com la comunicació via satèl·lit, per un cost molt superior i una complexitat d'integració fora de l'abast d'un TFG, i la ràdio convencional, que hauria requerit llicència d'operador i coneixements de radiocomunicacions que no aportaven cap avantatge rellevant respecte a LoRa per al cas d'ús plantejat. Tal com es detalla al Capítol 2, finalment no s'ha pogut completar la integració d'aquesta tecnologia en aquesta versió del projecte.

7.3 Elecció del mòdem 4G/SMS

Es va escollir el mòdem Huawei E3372 en mode HiLink per la seva doble funcionalitat: permet tant l'enviament i la recepció de missatges SMS com l'accés a dades mòbils 4G. Aquestes últimes són necessàries perquè el pont de WhatsApp pugui funcionar quan el dispositiu no disposa de connexió WiFi.

Es va descartar la integració d'un mòdul SIM independent connectat directament als pins GPIO (com un mòdul SIM800). Aquesta opció hauria requerit implementar tota la comunicació mitjançant comandes AT de baix nivell, sense oferir cap interfície de gestió, i no proporcionaria en cap cas accés a dades mòbils per a l'ús de WhatsApp, limitant el dispositiu únicament a SMS.

7.4 Elecció de pantalla, teclat físic, àudio i impressora

Es va optar per un teclat físic de poques tecles en lloc d'una pantalla tàtil per dos motius. Per una banda, el cost d'un teclat físic és molt inferior al d'una pantalla tàtil de qualitat suficient. Per l'altra, un nombre reduït de tecles físiques permet definir prou combinacions de navegació (mitjançant Tab, Shift+Tab i Espai, tal com es detalla al Capítol 8), sense necessitat que l'usuari hagi de fer gestos tàctils, que poden resultar menys intuïtius per a una persona gran.

Pel que fa a la impressora tèrmica, es va escollir davant l'opció de mostrar únicament els missatges en pantalla perquè resulta molt més intuïtiva i visual per a l'usuari final. Un missatge imprès en paper és una senyal física i immediata que ha arribat una comunicació nova. L'usuari no ha de saber navegar fins a un apartat concret de l'aplicació, seleccionar el missatge i desplaçar-se pel contingut en pantalla, accions que poden no ser evidents per a una persona gran poc familiaritzada amb interfícies digitals.

7.5 Decisions de programari

L'aplicació principal s'ha implementat en Qt6/C++ per tres motius: el framework Qt permet construir interfícies d'usuari intuïtives amb relativa facilitat, C++ ofereix el rendiment necessari per a un dispositiu amb recursos limitats com la Raspberry Pi, i l'estudiant ja disposava d'una petita experiència prèvia amb Qt (Capítol 2). Es van descartar alternatives com Python amb interfícies gràfiques (Tkinter o PyQt) o Electron, ja que no ofereixen el mateix rendiment ni el mateix control de baix nivell sobre el maquinari que ofereix C++.

La integració amb WhatsApp s'ha implementat mitjançant un procés Node.js independent en lloc de fer-ho directament en C++, ja que no es va trobar cap llibreria de C++ activa i mantinguda per a aquesta finalitat. Les llibreries de JavaScript disponibles, com *whatsapp-web.js*, es troben entre les més actualitzades i utilitzades per a aquest propòsit.

Per a la transcripció de veu es va escollir *faster-whisper* enfront d'altres opcions locals i de serveis de transcripció en línia. Els motors de transcripció locals alternatius provats van oferir una qualitat clarament inferior. Els serveis en línia, en canvi, es van descartar per dos motius: el risc de filtració de dades personals de l'usuari en enviar l'àudio a un servidor extern, i la dependència d'una connexió a internet i

d'un servei de tercers que, en cas de fallada, deixaria el sistema sense capacitat de transcripció.

7.6 Alternatives descartades

A banda de les alternatives ja comentades a les seccions anteriors, es resumeixen a continuació les principals opcions descartades durant el desenvolupament del projecte:

- **Integració de LoRa via GPIO:** tal com es detalla al Capítol 2, es van provar dos mòduls LoRa connectats directament als pins GPIO de la Raspberry Pi 5, sense que cap dels dos fos reconegut correctament pel sistema operatiu. Aquesta via es descarta en favor d'un futur mòdul LoRa en format USB (Capítol 12).
- **Generació de codi mitjançant *vibe coding*:** tal com es detalla al Capítol 4, es va provar de generar una primera versió del codi de manera majoritàriament automàtica amb eines d'intel·ligència artificial. Aquesta aproximació es va descartar per la baixa qualitat i mantenibilitat del codi resultant. Es va optar finalment per una implementació manual i acurada de tot el sistema.

Capítol 8

Anàlisi i disseny del sistema

8.1 Actors i casos d'ús

La Figura 8.1 mostra el diagrama de casos d'ús general del sistema TerraLink, amb els dos actors identificats al Capítol 6 (Persona Dependent i Cuidador) i els casos d'ús principals que hi interactuen.

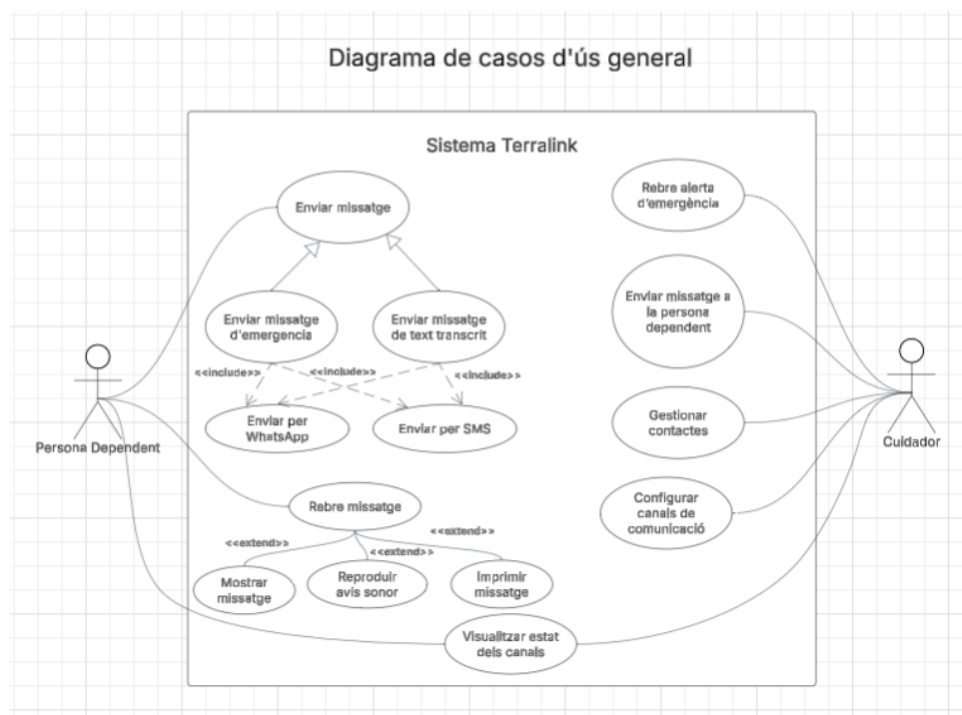


FIGURA 8.1: Diagrama de casos d'ús general de TerraLink

La Persona Dependent pot enviar missatges d'emergència, enviar missatges de text transcrit i rebre missatges. El Cuidador, a més de poder rebre alertes d'emergència i enviar missatges a la Persona Dependent, gestiona els contactes i configura els canals de comunicació disponibles. Ambdós actors poden, a més, visualitzar en tot moment l'estat dels canals de comunicació (RF8): a diferència de la resta de casos d'ús, no es tracta d'una acció que l'actor activa puntualment, sinó d'una icona amb indicador lluminós (LED) present de manera contínua a la interfície per a cada canal, motiu pel qual s'associa directament amb tots dos actors.

Els casos d'ús *Enviar missatge d'emergència* i *Enviar missatge de text transcrit* generalitzen un cas d'ús abstracte comú, *Enviar missatge*, del qual hereten l'associació amb l'actor Persona Dependent. La diferència essencial entre tots dos rau en els canals pels quals s'envia el missatge, modelada mitjançant relacions «include»: *Enviar missatge d'emergència* inclou obligatòriament l'enviament pel canal WhatsApp i pel canal SMS (i, en el futur, LoRa), de manera que l'alerta es propaga sempre per tots els canals disponibles; en canvi, *Enviar missatge de text transcrit* només inclou l'enviament per WhatsApp. Això reflecteix directament el requisit RNF2: el protocol d'emergència garanteix redundància de canals, mentre que l'enviament de missatges normals encara no en disposa (Capítol 10).

Els casos d'ús *Enviar missatge de veu* i *Enviar missatge de text transcrit* comparteixen un mateix flux intern: gravar l'àudio, transcriure'l a text, confirmar-lo i enviar-lo, tal com es mostra a la Figura 8.2.

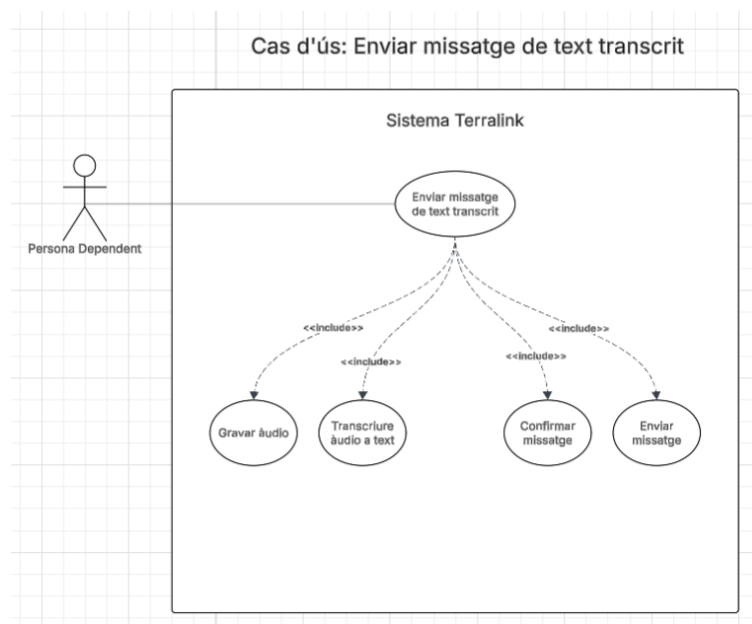


FIGURA 8.2: Cas d'ús: Enviar missatge de text transcrit, amb els sub-casos inclosos

De manera similar, el cas d'ús *Rebre missatge* inclou obligatòriament («include») el subcas *Mostrar missatge*, mentre que *Reproduir avís sonor* i *Imprimir missatge* en són extensions opcionals («extend»), condicionades respectivament a l'existència de sortida d'àudio i a que la impressora estigui activada. Aquesta distinció és important: mostrar el missatge és sempre necessari, mentre que l'avís sonor i la impressió depenen de la configuració i del maquinari disponible en cada moment (Figura 8.3).

8.2 Fitxes de casos d'ús

A continuació es detallen les fitxes dels quatre casos d'ús considerats més rellevants per al sistema.

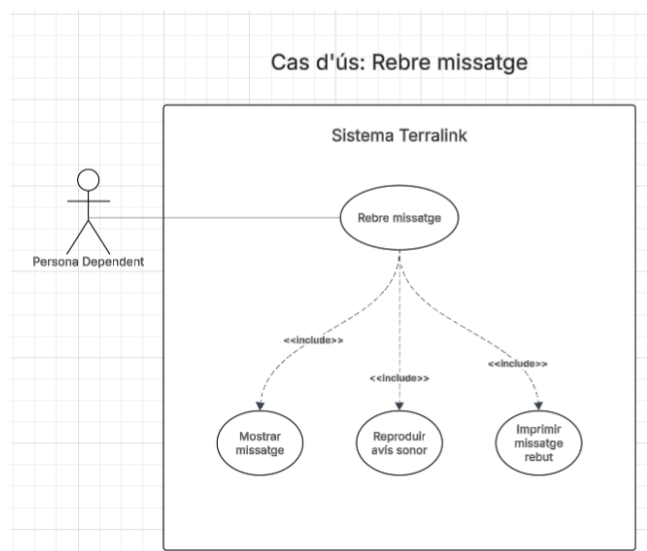


FIGURA 8.3: Cas d'ús: Rebre missatge, amb els subcasos inclosos

8.2.1 Fitxa 1. Enviar missatge d'emergència

Nom del cas d'ús	Enviar missatge d'emergència
Actor principal	Persona Dependent
Actors secundaris	Cuidador i altres contactes marcats com a emergència
Objectiu	Permetre que la Persona Dependent envii una alerta urgent a tots els seus contactes d'emergència de manera ràpida i senzilla.
Descripció	La Persona Dependent activa l'enviament d'un missatge d'emergència mitjançant el botó físic o una opció de la interfície. El sistema genera una alerta i l'envia simultàniament a tots els contactes marcats com a emergència, per tots els canals de comunicació disponibles en aquell moment.
Precondicions	El dispositiu TerraLink està encès. Hi ha almenys un contacte marcat com a contacte d'emergència. El sistema disposa d'almenys un canal de comunicació configurat.
Activació	La Persona Dependent prem el botó d'emergència o selecciona l'opció d'emergència a la pantalla.
Flux principal	1. La Persona Dependent activa l'opció d'emergència. 2. El sistema genera automàticament un missatge d'alerta. 3. El sistema identifica tots els contactes marcats com a emergència. 4. Per a cada contacte, el sistema envia el missatge simultàniament per tots els canals disponibles (WhatsApp i SMS actualment). 5. El sistema registra, per a cada canal, si l'enviament ha estat correcte o ha fallat. 6. El sistema mostra una confirmació a la Persona Dependent si com a mínim un canal ha confirmat l'enviament.
Fluxos alternatius	A1. Alguns canals no disponibles: si algun canal no respon o falla, el sistema continua intentant-ho pels altres canals sense aturar el procés.
Excepcions	E1. Cap canal disponible: si cap canal respon, el sistema informa l'usuari de l'error. E2. Cap contacte d'emergència configurat: si no hi ha cap contacte marcat com a emergència, el sistema no envia res i ho registra.
Postcondicions	El missatge d'emergència queda enviat a tots els contactes d'emergència pels canals que hagin respost correctament.
Prioritat	Alta
Requisits relacionats	RF3 (activació del protocol d'emergència), RNF2 (fiabilitat i redundància de canals), RNF1 (usabilitat per a persones grans)

TAULA 8.1: Fitxa del cas d'ús Enviar missatge d'emergència

8.2.2 Fitxa 2. Enviar missatge de text transcrit

Nom del cas d'ús	Enviar missatge de text transcrit
Actor principal	Persona Dependent
Actors secundaris	Cap
Objectiu	Permetre que la Persona Dependent envii un missatge escrit sense haver d'utilitzar un teclat, a partir de la transcripció automàtica d'un àudio.
Descripció	La Persona Dependent grava un missatge de veu. El sistema transcriu l'àudio a text, mostra el contingut transcrit perquè es pugui confirmar i finalment envia el missatge, per WhatsApp, al contacte seleccionat.
Precondicions	El dispositiu TerraLink està encès. El micròfon funciona correctament. Hi ha almenys un contacte configurat. El mòdul de transcripció està disponible.
Activació	La Persona Dependent prem el botó corresponent o selecciona l'opció d'enviar missatge de text transcrit a la interfície.
Flux principal	1. La Persona Dependent selecciona l'opció d'enviar missatge de text transcrit. 2. El sistema activa el micròfon i inicia la gravació. 3. La Persona Dependent dicta el missatge. 4. El sistema finalitza la gravació. 5. El sistema transcriu l'àudio a text. 6. El sistema mostra el text transcrit a la pantalla. 7. La Persona Dependent confirma el missatge. 8. El sistema envia el missatge per WhatsApp al contacte seleccionat. 9. El sistema mostra una confirmació d'enviament.
Fluxos alternatius	A1. Correcció o cancel·lació del missatge: si la Persona Dependent no accepta el text transcrit, pot cancel·lar l'enviament o tornar a gravar el missatge.
Excepcions	E1. Error de gravació: si el micròfon no funciona o no es detecta àudio, el sistema informa de l'error. E2. Error de transcripció: si no es pot generar el text, el sistema permet repetir la gravació. E3. WhatsApp no disponible: si el canal de WhatsApp no està connectat, el missatge queda marcat com a fallit; actualment el sistema no recorre automàticament a un canal alternatiu (Capítol 9).
Postcondicions	El missatge transcrit queda enviat al contacte seleccionat, o marcat com a fallit si WhatsApp no estava disponible.
Prioritat	Alta
Requisits relacionats	RF1 (enviament de missatges de veu transcrits a text), RNF1 (usabilitat per a persones grans)

TAULA 8.2: Fitxa del cas d'ús Enviar missatge de text transcrit

8.2.3 Fitxa 3. Rebre missatge

Nom del cas d'ús	Rebre missatge
Actor principal	Persona Dependent
Actors secundaris	Cuidador
Objectiu	Permetre que la Persona Dependent rebi missatges del Cuidador de manera clara i accessible, amb suport visual i físic.
Descripció	Quan el sistema rep un missatge enviat pel Cuidador, el mostra per pantalla i, si la impressora està activada, imprimeix el missatge rebut.
Precondicions	El dispositiu TerraLink està encès. Hi ha almenys un canal de recepció configurat. La pantalla està operativa.
Activació	El sistema rep un missatge provinent del Cuidador a través d'un dels canals disponibles (WhatsApp o SMS).
Flux principal	1. El sistema rep un missatge entrant. 2. El sistema identifica l'origen del missatge. 3. El sistema valida el contingut rebut. 4. El sistema mostra el missatge a la pantalla. 5. Si la impressora està activada, el sistema imprimeix el missatge rebut. 6. El sistema registra la recepció del missatge.
Fluxos alternatius	A1. Impressora desactivada: si la impressora no està activada, el sistema només mostra el missatge per pantalla.
Excepcions	E1. Missatge corrupte o incomplet: si el missatge no es pot interpretar correctament, el sistema mostra un avís d'error. E2. Error d'impressió: si la impressora falla, el sistema mostra el missatge per pantalla i registra l'error.
Postcondicions	El missatge queda mostrat i registrat. Si és possible, també queda imprès.
Prioritat	Mitjana-alta
Requisits relacionats	RF2 (recepció i impressió de missatges entrants), RNF1 (usabilitat per a persones grans)

TAULA 8.3: Fitxa del cas d'ús Rebre missatge

Nota: la fitxa original preveia un avís sonor en rebre un missatge, però finalment no s'ha pogut implementar per manca d'una targeta de so a la Raspberry Pi (els altaveus estaven adquirits, però sense targeta de so no s'hi podien connectar). Aquesta funcionalitat queda pendent com a treball futur (Capítol 12).

8.2.4 Fitxa 4. Configurar WhatsApp i contactes

Nom del cas d'ús	Configurar WhatsApp i contactes
Actor principal	Cuidador
Actors secundaris	Cap
Objectiu	Permetre que el Cuidador vinculi el compte de WhatsApp i gestioni els contactes del sistema TerraLink.
Descripció	El Cuidador accedeix a la pantalla de configuració mitjançant la combinació secreta de tecles (Annex A). Des d'allà pot vincular un compte de WhatsApp escanejant un codi QR, i gestionar els contactes: afegir-los manualment, importar-los des de WhatsApp, modificar-los o eliminar-los.
Precondicions	El dispositiu TerraLink està encès. El Cuidador coneix la combinació secreta d'accés a la configuració.
Activació	El Cuidador accedeix a la pantalla de configuració i selecciona "QR WhatsApp" o "Contactes".
Flux principal	1. El Cuidador accedeix a la pantalla de configuració. 2. Per vincular WhatsApp: selecciona "QR WhatsApp", el sistema mostra el codi QR, i el Cuidador l'escaneja des del mòbil que es vulgui vincular. 3. Per gestionar contactes: selecciona "Contactes" i pot afegir-ne un de nou, importar-los automàticament des de WhatsApp, modificar-ne un existent, o eliminar-lo. 4. El sistema desa els canvis fets.
Fluxos alternatius	A1. Importació de contactes: en importar des de WhatsApp, el sistema filtra automàticament els contactes que no corresponguin a un número de telèfon vàlid (10-12 dígitos).
Excepcions	E1. WhatsApp no vinculat: si s'intenta importar contactes sense haver vinculat abans un compte de WhatsApp, el sistema ho indica i no fa res. E2. Dades de contacte incompletes: si el nom o el número introduïts manualment no són vàlids, el sistema demana corregir-los.
Postcondicions	El compte de WhatsApp queda vinculat i/o els contactes queden actualitzats al sistema.
Prioritat	Mitjana
Requisits relacionats	RF4 (gestió de contactes), RF5 (importació de contactes des de WhatsApp), RF6 (vinculació del compte de WhatsApp mitjançant codi QR)

TAULA 8.4: Fitxa del cas d'ús Configurar WhatsApp i contactes

Nota: marcar un contacte com a contacte d'emergència no és encara possible des d'aquesta pantalla; actualment cal editar directament el fitxer de contactes (Capítol 12).

8.3 Model de dades

El model de dades de TerraLink es construeix al voltant de dues entitats principals: *Contacte* i *Missatge*. La columna "DADES" del diagrama de classes complet [14] (Figura 8.6) mostra les relacions entre aquestes entitats.

L'entitat *Contacte* emmagatzema l'identificador, el nom, el número de telèfon i l'adreça LoRa (si escau) de cada contacte del sistema, incloent un indicador de si es tracta d'un contacte d'emergència. Aquesta entitat es gestiona mitjançant el *RepositoriContactes*, implementat com a Singleton amb persistència en un fitxer JSON.

L'entitat `Missatge` és una classe abstracta de la qual hereten `MissatgeWhatsApp`, `MissatgeSMS` i `MissatgeLoRa`, cadascuna amb els atributs propis del canal corresponent. Per exemple, `numeroWA` per a WhatsApp, o `nodeDestinaci` i la mida màxima de paquet per a LoRa. Cada missatge manté el seu estat mitjançant l'enumeració `EstatMissatge` (*Esborrany*, *Enviant*, *Enviat*, *Rebut*, *Fallit*, *Imprès*), i està associat a un `Contacte` destinatari mitjançant una relació d'un a molts: un contacte pot ser destinatari de diversos missatges. Els missatges es gestionen mitjançant el `RepositoriMissatges`, també Singleton, tot i que en aquest cas sense persistència a disc (Capítol 9).

Les instàncies concretes de `Missatge` es creen mitjançant la fàbrica `MissatgeFactory` (patró Factory), que instancia la subclasse adequada segons el canal de destí especificat.

8.4 Diagrama de desplegament

La Figura 8.4 mostra el diagrama de desplegament del sistema: com es distribueix el programari sobre el maquinari físic i com es comuniquen entre si els diferents dispositius.

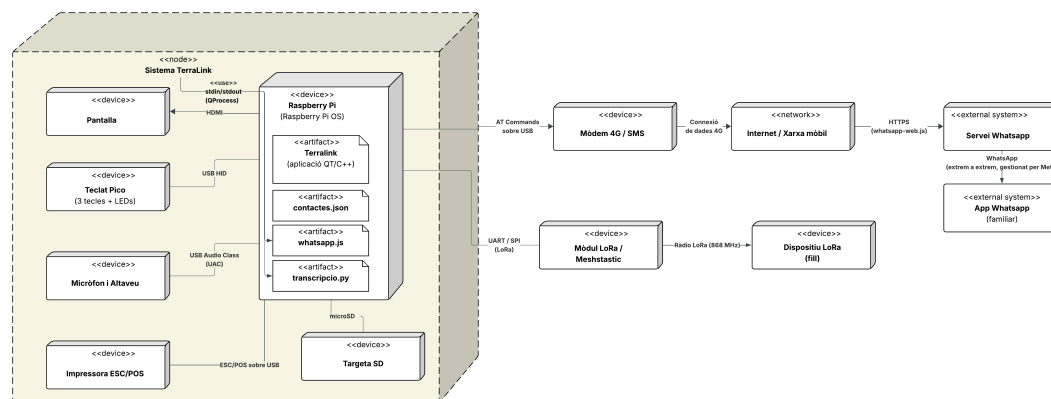


FIGURA 8.4: Diagrama de desplegament de TerraLink

Tot el programari de TerraLink s'executa sobre un únic node, la Raspberry Pi, que allotja quatre artefactes principals: l'aplicació TerraLink (Qt/C++), el fitxer `contactes.json`, i els dos processos externs que TerraLink llança i controla mitjançant `QProcess`: el pont `whatsapp.js` (Node.js) i l'script `transcripcio.py` (Python), amb qui es comunica per `stdin/stdout`. Aquest node es connecta a diversos dispositius perifèrics: una pantalla (via HDMI), un teclat Pico de 3 tecles amb LEDs (via USB HID), un micròfon i altaveu (via USB Audio Class), una impressora ESC/POS (via USB) i una targeta microSD per a l'emmagatzematge.

Cap a l'exterior, la Raspberry Pi es comunica amb un mòdem 4G/SMS mitjançant comandes AT sobre USB. Aquest, mitjançant una connexió de dades 4G, accedeix a Internet/xarxa mòbil per arribar al servei de WhatsApp (connexió HTTPS/WebSocket gestionada per la llibreria `whatsapp-web.js`) i, des d'allà, a l'aplicació de WhatsApp del familiar (comunicació extrem a extrem xifrada, gestionada per Meta). De manera independent, el node es connecta també a un mòdul LoRa/Meshtastic mitjançant UART/SPI, que es comunicaria per ràdio LoRa (868 MHz) amb el dispositiu

LoRa del familiar. Aquesta funcionalitat encara està pendent d'integració, tal com es detalla al Capítol 2.

8.5 Diagrama de components

La Figura 8.5 mostra el diagrama de components de TerraLink: com es descompon l'aplicació en mòduls funcionals i les dependències que hi ha entre ells.

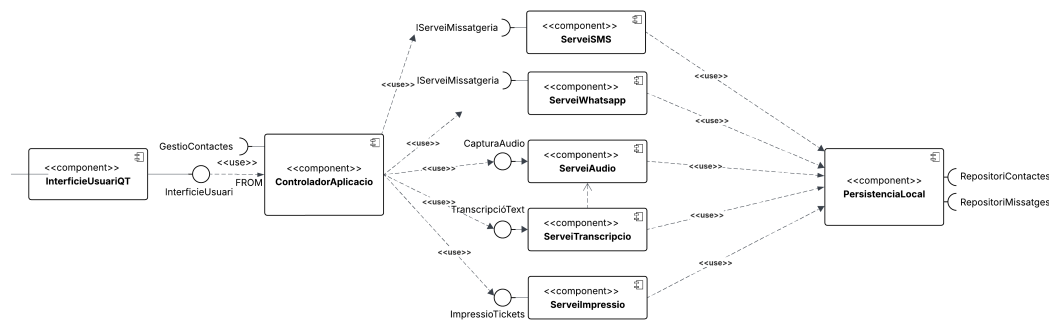


FIGURA 8.5: Diagrama de components de TerraLink

El component `InterficieUsuariQT` exposa una interfície `InterficieUsuari` que consumeix el `ControladorAplicacio`, el qual actua com a punt central de coordinació i exposa, al seu torn, la interfície `GestioContactes` (afegir, modificar, eliminar i marcar contactes com a emergència). El propi `ControladorAplicacio` assumeix també el monitoratge de l'estat dels canals de comunicació (RF8), sense necessitat d'un component separat. A banda d'això, fa ús de quatre components de servei especialitzats: `ServeiWhatsapp` i `ServeiSMS` (que implementen tots dos la interfície genèrica `IServeiMissatgeria`, seguint el patró Strategy descrit al Capítol 9), `ServeiAudio` (captura d'àudio), `ServeiTranscripcio` (transcripció de text, que depèn al seu torn de `ServeiAudio`) i `ServeiImpressio` (impressió de tiquets). Tots aquests components fan ús, al seu torn, del component `PersistenciaLocal`, que exposa les interfícies `RepositoriContactes` i `RepositoriMissatges`. Així es centralitza tot l'accés a dades persistents del sistema. Cal remarcar que `ServeiWhatsapp` també fa ús de `PersistenciaLocal` directament en el cas concret d'importació de contactes des de WhatsApp (RF5).

La Figura 8.6 mostra el diagrama de classes complet del sistema, organitzat en cinc columnes segons la seva responsabilitat: interfície/vista, control, serveis i patrons, adaptadors, i dades.

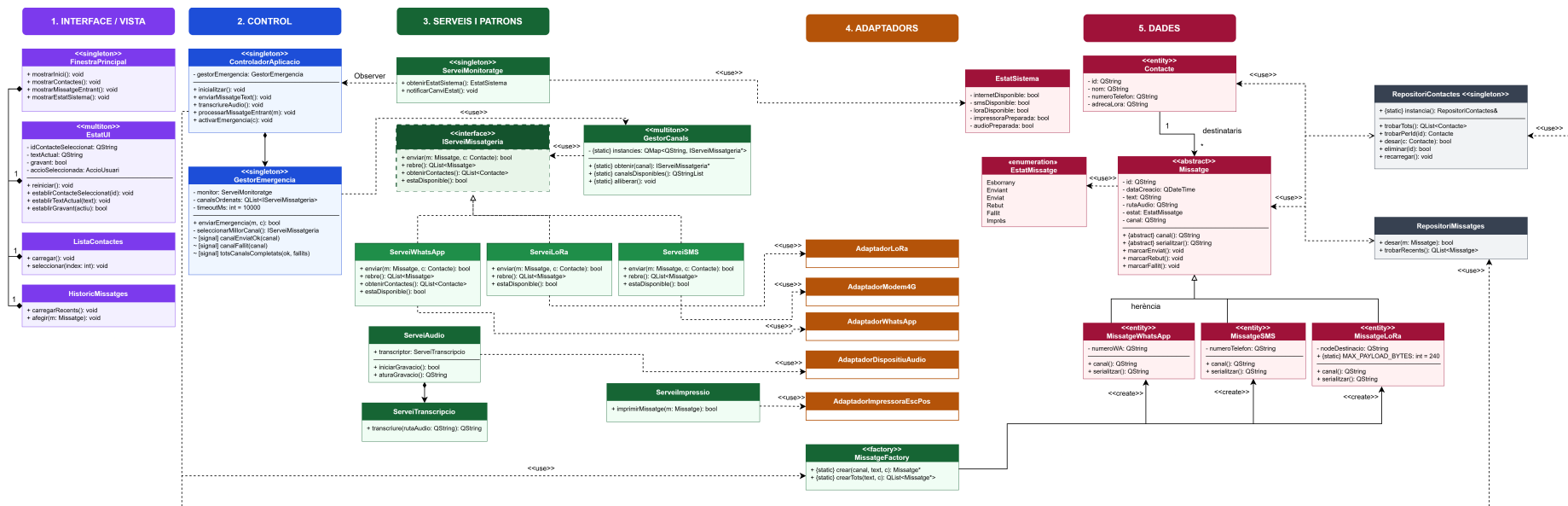


FIGURA 8.6: Diagrama de classes de TerraLink

Capítol 9

Implementació i proves

9.1 Implementació del maquinari

La implementació del maquinari s'ha basat en la integració dels components descrits al Capítol 8, seleccionats per la seva fiabilitat, cost i compatibilitat amb la Raspberry Pi (Capítol 7).

9.1.1 Raspberry Pi 5 (unitat central)

- Instal·lació de Raspberry Pi OS a la targeta microSD donada per Raspberry Pi (Capítol 2).
- Configuració dels ports USB per al mòdem 4G, el teclat Pico, la impressora tèrmica i el micròfon/altaveu.
- Instal·lació de l'entorn Qt6, de Node.js (per al pont de WhatsApp) i de Python amb *faster-whisper* (per a la transcripció de veu).

9.1.2 Pantalla

- Connexió via HDMI, tal com es mostra al diagrama de desplegament (Capítol 8).
- Interfície implementada amb Qt Widgets, amb botons grans i navegació mitjançant Tab/Shift+Tab/Espai per facilitar-ne l'ús a persones grans.

9.1.3 Teclat Raspberry Pico (3 tecles + LEDs)

- Connexió a la Raspberry Pi via USB HID, emulant les pulsacions de Tab, Shift+Tab i Espai que la interfície Qt interpreta per a la navegació.
- Ús dels LEDs per indicar visualment l'element de la interfície seleccionat en cada moment.

9.1.4 Mòdem 4G Huawei E3372

- Connexió via USB en mode HiLink, exposant una interfície de gestió web accessible a <http://192.168.8.1>.

- Comunicació amb aquesta interfície mitjançant peticions HTTP amb cos XML per enviar i rebre SMS (Secció 9.3), i per proporcionar accés a dades mòbils 4G quan la Raspberry Pi no disposa de connexió WiFi.

9.1.5 Micròfon

- Dispositiu d'àudio USB connectat via USB Audio Class (UAC), identificat pel sistema com a `hw:3,0`.
- Gravació mitjançant l'ordre `arecord` en format PCM de 16 bits a 16kHz, mono.

Cal remarcar que l'altaveu, tot i haver estat adquirit (Capítol 2), no s'ha arribat a connectar al dispositiu final per manca d'una targeta de so de sortida i de ports USB lliures. Per aquest motiu, el sistema no reproduïx cap so (per exemple, un avís sonor en rebre un missatge, Capítol 8), i aquesta funcionalitat queda pendent com a treball futur (Capítol 12).

9.1.6 Impressora tèrmica ESC/POS

- Connexió via USB, accessible pel sistema com a dispositiu `/dev/usb/lp0`.
- Comunicació directa mitjançant l'escriptura de seqüències de bytes ESC/POS (Capítol 5), sense necessitat de cap controlador (*driver*) addicional.

9.1.7 Mòdul LoRa i bateria

Tal com ja s'apuntava com a risc possible al Capítol 2, la integració d'un mòdul LoRa directament als pins GPIO de la Raspberry Pi 5 finalment no s'ha pogut completar. Es van adquirir dos mòduls LoRa diferents per provar-ho: el primer era incompatible amb la placa; el segon, tot i ser-hi compatible sobre el paper, no s'ha aconseguit que el sistema operatiu el reconegués correctament malgrat diverses proves de configuració. Per aquest motiu, s'ha decidit **descartar temporalment la integració de LoRa** en aquesta versió del projecte. La seva implementació queda com a treball futur (Capítol 12), previsiblement mitjançant un mòdul LoRa en format USB en lloc de GPIO, cosa que evitaria els problemes de compatibilitat de maquinari detectats.

Cal remarcar també que la Raspberry Pi 5 disposa d'un nombre limitat de ports USB, actualment tots ocupats pel mòdem 4G, el teclat, el micròfon i la impressora. Per afegir en el futur un mòdul LoRa USB caldria adquirir un *hub* USB alimentat (per exemple, un hub PCIe a USB 3.2 Gen1 per a Raspberry Pi 5). Aquest element encara no s'ha pogut adquirir per motius pressupostaris, i es contempla també com a treball futur.

El dispositiu tampoc disposa actualment de bateria pròpia. Ambdós elements (LoRa i bateria) es contemplen com a treball futur (Capítol 12).

9.2 Implementació del programari

El programari de TerraLink s'ha desenvolupat principalment en **Qt6/C++**, complementat amb un procés **Node.js** per a la integració amb WhatsApp i un script en **Python** per a la transcripció de veu (Capítol 7 en justifica l'elecció).

A continuació es mostra un vídeo explicant l'organització del codi (aplicació Qt6/C++, pont de WhatsApp en Node.js i script de transcripció en Python) i com es comuniquen entre si els tres processos.

Vídeo d'explicació del codi: https://drive.google.com/file/d/1xcajt8sJo7ip7dPVMHANmon13hMWt7q_/view?usp=sharing

9.2.1 Estructura general del programari

El sistema s'organitza en capes seguint els patrons de disseny descrits al Capítol 8 (Singleton, Multiton, Factory, Strategy, Adapter i Observer). La Taula 9.1 en resumeix els mòduls principals.

Mòdul	Funció
ControladorAplicacio	Coordinació central de l'aplicació (Singleton).
GestorCanals	Gestió de les instàncies dels serveis de missatgeria per canal (Multiton).
ServeiWhatsApp, ServeiSMS, ServeiLoRa	Implementacions concretes del canal de comunicació (Strategy).
GestorEmergencia	Enviament simultani (<i>broadcast</i>) del missatge d'emergència per tots els canals disponibles.
MissatgeFactory	Creació de la subclasse de <i>Missatge</i> adequada segons el canal (Factory).
RepositoriContactes, RepositoriMissatges	Persistència de contactes (JSON) i missatges en memòria (Singleton).
ServeiAudio, ServeiTranscripcio	Gravació de veu i transcripció a text amb <i>faster-whisper</i> .
ServeiImpressio	Impressió de missatges i imatges via ESC/POS.
ServeiMonitoratge	Monitoratge de l'estat del sistema (Observer, via signals/slots de Qt).

TAULA 9.1: Mòduls principals del programari TerraLink

9.2.2 Flux de funcionament del programari

1. Inicialització del *ControladorAplicacio*: arrencada del procés Node.js de WhatsApp, inici del *polling* de SMS i impressió de la capçalera inicial.
2. Espera d'accions de l'usuari mitjançant el teclat Pico (navegació Tab/Shift+Tab/Espai).
3. Captura del missatge de veu i transcripció a text mitjançant *faster-whisper*.
4. Confirmació del text transcrit a la pantalla per part de l'usuari.
5. Enviament del missatge pel canal seleccionat (WhatsApp, prioritàriament) o *broadcast* a tots els canals en cas d'emergència.
6. Registre del missatge al *RepositoriMissatges* i, si escau, impressió del missatge rebut.

9.3 Integració dels canals de comunicació

9.3.1 WhatsApp

- Integració mitjançant un procés Node.js [15] independent (*whatsapp.js*), que utilitza la llibreria *whatsapp-web.js* (Capítol 5).
- Comunicació entre l'aplicació Qt i el procés Node.js via `stdin/stdout`: la Qt envia comandes com `SEND:numero:text` o `GET_CONTACTS`, i Node respon amb `SEND_OK/SEND_ERROR`, el codi QR en format ASCII art, o els missatges entrants prefixats amb `INCOMING:`.

9.3.2 SMS

- Enviament i recepció mitjançant l'API REST HiLink del mòdem Huawei E3372, mitjançant peticions HTTP amb cos XML (Capítol 5).
- La recepció de nous SMS es fa per *polling*, comprovant cada 30 segons si hi ha missatges nous a la safata d'entrada del mòdem.

9.3.3 LoRa

Tal com s'ha explicat a les seccions anteriors, aquest canal no s'ha pogut integrar en aquesta versió del projecte.

9.3.4 Impressió ESC/POS

- Enviament directe de seqüències de bytes ESC/POS al dispositiu `/dev/usb/lp0`: inicialització, activació/desactivació de negreta, tall de paper i alimentació de línies.
- Suport per a la impressió d'imatges (JPEG/PNG), convertides a format monocrom i enviades com a dades ràster ESC/POS.

9.4 Gestió de redundància

El sistema implementa dues estratègies diferents de gestió de canals, segons el context:

- **Emergències:** el `GestorEmergencia` envia el missatge d'emergència **simultàniament per tots els canals disponibles** (WhatsApp, SMS i LoRa), en lloc de provar-los seqüencialment. Es considera que l'emergència s'ha comunicat correctament si com a mínim un canal ha confirmat l'enviament.
- **Missatges normals:** actualment, l'enviament d'un missatge de veu transcrit (RF1) es fa únicament pel canal de WhatsApp. Si aquest no està disponible, el missatge queda marcat com a fallit, sense recórrer automàticament a un canal alternatiu.

Aquesta diferència de comportament és una limitació identificada del sistema actual: el requisit RNF2 (fiabilitat i redundància de canals) es compleix plenament per al cas de les emergències, però no encara per als missatges normals. Implementar un

mecanisme de *fallback* automàtic (WhatsApp → SMS) per als missatges normals es planteja com a millora de cara al treball futur (Capítol 12).

9.5 Proves unitàries

A diferència de les proves d'integració (Secció 9.6), que validen el sistema complet un cop connectades totes les capes, les proves unitàries d'aquest projecte no s'han realitzat mitjançant un framework de testing automatitzat (com `QtTest`). S'han fet de manera manual i incremental, classe per classe, a mesura que s'anava implementant cada funcionalitat. Aquest enfocament és coherent amb la metodologia de treball descrita al Capítol 3: cada peça del sistema es validava abans d'integrar-la amb la següent, evitant acumular funcionalitats no verificades.

La tècnica de verificació emprada ha estat, majoritàriament, la inserció de missatges de traça amb `qDebug()` en punts crítics del codi, combinada amb l'observació manual del comportament resultant (a la consola "Application Output" de Qt Creator o al terminal, en el cas dels processos Node.js). La Taula 9.2 recull els casos més rellevants de proves unitàries realitzades durant el desenvolupament.

Component provat	Què es verificava	Tècnica de verificació
AdaptadorWhatsApp (gestió del procés Node.js)	Que el procés s'iniciés correctament i que els canvis d'estat (<code>APP_READY</code> , recepció del QR, desconnexió) es detectessin correctament a partir de la sortida estàndard del procés.	<code>qDebug()</code> als slots <code>enCanviarEstat()</code> i <code>enLlegirSortida()</code> , observant línia per línia la sortida del procés Node abans d'integrar-ho amb la interfície gràfica.
GestorEmergencia (mètode <code>enviarEmergencia()</code>)	Que el broadcast a tots els canals disponibles funcionés correctament, distingint entre canals no disponibles i canals que fallaven en l'enviament.	<code>qDebug()</code> per cada canal ("Canal enviat OK", "Canal no disponible", "Canal fallat") i recompte final de fallits, provat inicialment amb tots els adaptadors retornant <code>false</code> de manera controlada.
Enviament de missatges de WhatsApp (<code>SEND_OK</code> / <code>SEND_ERROR</code>)	Que la confirmació d'enviament arribés correctament abans de donar per fet l'enviament, detectant i corregint una condició de cursa entre la lectura del procés i l'espera de resposta.	Flags interns (<code>m_esperantResposta</code> , <code>m_ultimEnviamentOk</code>) i <code>qDebug()</code> per confirmar l'ordre d'execució real dels events.
ServeiWhatsApp (mètode <code>obtenirContactes()</code>)	Que el JSON de contactes rebut del procés Node s'acumulés i es parsegés correctament, especialment quan arribava fragmentat en múltiples lectures del <code>QProcess</code> .	<code>qDebug()</code> mostrant la mida del <i>buffer</i> acumulat i si el JSON resultant era un array vàlid, abans de comprovar que els contactes apareixien correctament a la interfície.

TAULA 9.2: Exemples de proves unitàries realitzades durant el desenvolupament

Aquest enfocament de proves manuals no ofereix la repetibilitat ni la cobertura automàtica d'un framework de testing formal, però ha permès detectar i corregir errors

concrets durant el desenvolupament. Per exemple, la condició de cursa en la confirmació d'enviament de WhatsApp o la fragmentació del JSON de contactes, abans d'integrar cada component amb la resta del sistema. Com a limitació reconeguda, la manca de proves automatitzades (relacionada també amb la manca de control de versions comentada al Capítol 3) es planteja com una millora a incorporar en futures iteracions del projecte.

9.6 Proves d'integració

S'han realitzat proves en diferents escenaris:

9.6.1 Proves de comunicació

- Enviament i recepció de missatges de WhatsApp de cap a cap, incloent la vinculació inicial mitjançant el codi QR i l'enviament d'un missatge d'emergència a diversos contactes simultàniament.
- Enviament i recepció de missatges SMS reals mitjançant el mòdem Huawei E3372 amb targeta SIM activa.
- Verificació que, quan el canal de WhatsApp no estava disponible (procés Node.js aturat), el missatge de veu quedava correctament marcat com a fallit en lloc de bloquejar l'aplicació.

9.6.2 Proves d'ús

- Test amb persones grans per validar la simplicitat de la interfície.
- Proves de gravació i transcripció de veu.
- Test de la impressora amb text i imatges.

Durant aquestes proves es va detectar que alguns teclats USB externs generaven múltiples informes HID per una sola pulsació física de tecla, cosa que trencava la detecció fiable de la combinació secreta d'accés a la pantalla de configuració (Annex A). Es va solucionar aplicant un filtre de *debounce* per programari a nivell de la interfície: el sistema ignora repeticions de la mateixa tecla si es detecten en un interval inferior a 150 mil·lisegons. Aquest llindar és prou curt per no interferir amb la navegació normal (Tab/Shift+Tab), però suficient per filtrar els duplicats generats pel maquinari.

Les proves relacionades amb la comunicació LoRa en zones sense cobertura i amb l'autonomia de bateria no s'han pogut realitzar en aquesta versió del projecte, ja que cap dels dos elements s'ha integrat finalment al sistema (Capítol 2). Ambdues proves queden pendents per a quan aquestes funcionalitats s'implementin com a treball futur (Capítol 12).

Capítol 10

Implantació i resultats

10.1 Procés d'implantació

La posada en marxa de TerraLink sobre un dispositiu nou segueix els passos següents:

1. **Instal·lació del sistema operatiu:** flashear Raspberry Pi OS a la targeta microSD i realitzar la configuració inicial (usuari, xarxa WiFi si escau, activació de SSH).
2. **Instal·lació de dependències:** instal·lar Qt6 i les eines de compilació (`qt6-base-dev`, `cmake`, `ninja`), Node.js (per al pont de WhatsApp) i Python 3 amb *faster-whisper* (`pip install faster-whisper`).
3. **Desplegament del codi:** copiar el projecte TerraLink (codi C++, l'script `whatsapp.js` i l'script de transcripció Python) al directori de treball de la Raspberry Pi
4. `/home/paudomenech/TerraLink/`, i instal·lar les dependències de Node.js (`npm install`) dins d'aquest directori.
5. **Compilació:** compilar l'aplicació Qt6/C++ amb CMake i Ninja des de Qt Creator o des de línia de comandes.
6. **Connexió del maquinari perifèric:** connectar per USB el mòdem 4G, la impressora tèrmica, el micròfon i el teclat Pico, i connectar la pantalla via HDMI. L'altaveu adquirit no s'ha pogut connectar per manca d'una targeta de so de sortida (Capítol 9).
7. **Configuració de l'arrencada automàtica:** es configura un servei `systemd` (`terralink.service`) que arrenca automàticament l'aplicació TerraLink en engegar la Raspberry Pi, de manera que l'usuari final no ha de fer cap acció manual per posar en marxa el dispositiu.
8. **Configuració inicial per part del Cuidador:** un cop engegat el sistema per primera vegada, el Cuidador accedeix a la pantalla de configuració (Capítol 8) per vincular el compte de WhatsApp mitjançant el codi QR i importar o donar d'alta els contactes, marcant els que corresponguin com a contactes d'emergència.

Aquest procediment es correspon amb l'escenari de comercialització esmentat al

Capítol 2: la instal·lació i configuració del dispositiu la duria a terme un tècnic o un familiar de la Persona Dependent, seguint un manual de configuració, sense que aquesta hagi d'intervenir-hi en cap moment.

10.2 Resultats

A continuació es mostra una demostració en vídeo del funcionament del dispositiu TerraLink, incloent l'enviament d'un missatge de veu transcrit, la recepció d'un missatge amb la seva impressió i l'activació del protocol d'emergència.

Vídeo de demostració: <https://drive.google.com/file/d/1asaAzjz388vbFXIXBZngfc2v4TkPYg3G/view?usp=sharing>

10.2.1 Assoliment dels requisits funcionals

La Taula 10.1 mostra el grau d'assoliment de cadascun dels requisits funcionals definits al Capítol 6.

ID	Requisit	Estat
RF1	Enviament de missatges de veu transcrits a text	Complert
RF2	Recepció i impressió de missatges entrants	Complert
RF3	Activació del protocol d'emergència	Complert
RF4	Gestió de contactes	Complert
RF5	Importació de contactes des de WhatsApp	Complert
RF6	Vinculació del compte de WhatsApp mitjançant codi QR	Complert
RF7	Enviament i recepció de missatges per SMS	Complert

TAULA 10.1: Assoliment dels requisits funcionals

Tots els requisits funcionals s'han assolit satisfactòriament. La comunicació via LoRa no formava part dels requisits definits al Capítol 6 (RF1-RF7 no la inclouen com a requisit obligatori), tot i que sí que es va contemplar inicialment com a canal addicional i finalment no s'ha pogut integrar (Capítol 2).

10.2.2 Assoliment dels requisits no funcionals

La Taula 10.2 mostra el grau d'assoliment dels requisits no funcionals.

ID	Requisit	Estat
RNF1	Usabilitat per a persones grans	Complert
RNF2	Fiabilitat i redundància de canals	Parcialment complert
RNF3	Privacitat de les dades	Complert
RNF4	Disponibilitat contínua	Parcialment complert
RNF5	Facilitat d'expansió	Complert

TAULA 10.2: Assoliment dels requisits no funcionals

Dos requisits no funcionals només s'han assolit parcialment:

- **RNF2 (Fiabilitat i redundància de canals):** aquest requisit es compleix plenament per al protocol d'emergència, que fa *broadcast* simultani a tots els canals disponibles (Capítol 9). L'enviament de missatges normals (RF1), en canvi, es fa únicament pel canal de WhatsApp, sense recórrer automàticament a SMS si aquest falla. Aquesta millora es planteja com a treball futur (Capítol 12).
- **RNF4 (Disponibilitat contínua):** el sistema arrenca automàticament i funciona de manera contínua mentre hi ha subministrament elèctric, gràcies al servei `systemd` configurat. Tanmateix, com que el dispositiu no disposa de bateria pròpia (Capítol 2), un tall de corrent deixa l'usuari sense cap via de comunicació, incloent-hi les d'emergència, fins que es restableixi l'alimentació. Aquesta limitació es considera prioritària de resoldre en treball futur.

10.3 Protecció de dades i normativa aplicable

Ampliant el que ja s'ha exposat a nivell de viabilitat al Capítol 2, en aquesta secció es detallen els aspectes pràctics d'implantació relacionats amb la protecció de dades personals (RGPD/LOPD) i la LSSICE.

Totes les dades personals gestionades pel sistema (contactes) resideixen exclusivament a la targeta microSD de la Raspberry Pi, en un fitxer JSON local (`contactes.json`), sense cap còpia ni sincronització amb servidors externs. Físicament, la targeta microSD es troba inserida al propi dispositiu TerraLink, ubicat al domicili de la Persona Dependent, i únicament hi tenen accés físic la pròpia Persona Dependent i el Cuidador responsable de la configuració. No hi ha cap altra part (fabricant, desenvolupador o tercer) amb accés remot a aquestes dades, ja que el dispositiu no exposa cap servei de xarxa accessible des de l'exterior.

En cas de baixa del servei, canvi d'usuari final o retirada definitiva del dispositiu, l'esborrat de les dades personals es podria dur a terme substituint el fitxer `contactes.json` per un de buit o reformatant la targeta microSD, sense que calgui cap gestió addicional amb tercers, ja que no existeix cap còpia externa de les dades.

Pel que fa a la LSSICE, es reitera el que ja s'ha exposat al Capítol 2: en l'estat actual del projecte, desenvolupat com a TFG i sense finalitat comercial, aquesta normativa no és aplicable. En un futur escenari de comercialització, caldria revisar aquest apartat amb assessorament legal específic.

Capítol 11

Conclusions

11.1 Resum dels objectius i assoliment

L'objectiu de TerraLink era oferir a una persona gran un dispositiu de comunicació més actual que els mitjans tradicionals. Havia de permetre-li mantenir el contacte amb la seva família –rebent i responent missatges i fotografies a través de qualsevol canal disponible– i, sobretot, disposar d'una via garantida per enviar un avís d'emergència quan calgués, amb la seguretat que aquest avís s'enviaria sí o sí. Tot això calia aconseguir-ho amb una interfície molt intuïtiva, adaptada a una persona sense experiència amb dispositius electrònics.

Aquest objectiu s'ha assolit en gran part. El sistema permet enviar missatges de veu transcrits a text, rebre'ls i imprimir-los, i activar un protocol d'emergència que fa arribar l'avís simultàniament per tots els canals disponibles (Capítol 9). Això garanteix la comunicació en cas d'emergència de manera molt més robusta que un enviament per un sol canal. La interfície, basada en un teclat físic de només tres tecles i una navegació senzilla, s'ha dissenyat íntegrament pensant en la facilitat d'ús per a una persona gran (RNF1).

Tanmateix, la visió inicial de "gaudir del grup de família" veient fotografies i responent amb normalitat per qualsevol canal no s'ha assolit de manera tan completa com es plantejava. El sistema permet rebre i imprimir fotografies, però no ofereix encara una experiència de navegació o galeria de missatges rica. Tampoc s'ha implementat encara el *fallback* automàtic entre canals per als missatges normals, més enllà de les emergències (Capítol 10).

Amb tot, cal remarcar que aquestes mancances no són limitacions estructurals del sistema, sinó funcionalitats deliberadament deixades fora de l'abast d'aquest TFG. El disseny de l'arquitectura (Capítol 8) s'ha pensat expressament perquè aquestes ampliacions es puguin incorporar sense haver de redissenyar el sistema: el patró *Strategy* aplicat a `GestorCanals` permet afegir un nou canal de comunicació (com LoRa) implementant-ne únicament l'adaptador corresponent, sense tocar la resta de la lògica de l'aplicació; la fàbrica `MissatgeFactory` i la jerarquia d'herència de `Missatge` permeten incorporar nous tipus de missatge (per exemple, amb suport nadiu per a imatges) simplement afegint una subclasse nova; i la separació en capes del diagrama de components (interfície, control, serveis, adaptadors, dades) aïlla els canvis de maquinari o de canal a la capa d'adaptadors, sense que es propaguin cap a la interfície d'usuari. Aquest requisit d'extensibilitat (RNF5) era, de fet, un objectiu de disseny explícit des de l'inici del projecte, precisament perquè s'esperava que no

tot l'abast inicial es pogués cobrir dins el temps d'un TFG.

11.2 Principals resultats obtinguts

El resultat principal del projecte és un dispositiu funcional, construït amb maquinari de baix cost (Capítol 2), capaç d'enviar i rebre missatges per WhatsApp i SMS, transcriure missatges de veu, imprimir-los en paper i activar un protocol d'emergència multicanal.

Comparat amb altres solucions existents al mercat, com certs dispositius que imprimeixen automàticament els missatges i fotografies de WhatsApp rebuts però que només permeten respondre amb un simple sí o no i que deixen de funcionar completament si no hi ha connexió WiFi, TerraLink aporta un salt qualitatiu rellevant. La comunicació no depèn d'un únic mitjà (permet WhatsApp per WiFi o dades mòbils, i SMS com a alternativa), i el protocol d'emergència es basa en un enviament simultani per tots els canals disponibles, en lloc de dependre d'una única via que pugui fallar en el moment més crític.

11.3 Aportacions personals

Tot el disseny i la implementació del sistema són fruit del treball de l'estudiant: des dels diagrames de classes, casos d'ús i desplegament (Capítol 8) fins a la totalitat del codi de l'aplicació. Destaca especialment la decisió d'integrar WhatsApp mitjançant un pont extern en Node.js, presa després de no trobar cap llibreria de C++ activa i mantinguda per a aquesta finalitat (Capítol 7), així com el disseny de tot el flux de transcripció de veu a text mitjançant *faster-whisper*. L'arquitectura completa del sistema, amb els patrons de disseny aplicats (Singleton, Multiton, Factory, Strategy i Observer, Capítol 8), és una aportació pròpia i original de l'estudiant, sense partir de cap sistema o codi previ existent.

11.4 Aprenentatge realitzat

Aquest projecte ha suposat un aprenentatge significatiu en diversos àmbits. A nivell tècnic, destaca l'aprenentatge en profunditat del framework Qt6/C++, que ha demostrat ser una eina molt potent i versàtil per construir interfícies gràfiques senzilles i alhora integrar-les amb maquinari extern (mòdems, impressores, dispositius d'àudio). També ha suposat un aprenentatge important treballar amb tecnologies i protocols amb els quals l'estudiant no tenia experiència prèvia, com l'API HiLink de Huawei, el protocol ESC/POS o la integració de processos externs (Node.js, Python) des d'una aplicació C++ mitjançant `QProcess`.

A nivell metodològic, el projecte també ha posat de manifest la importància de disciplines que no es van aplicar des del principi, com el control de versions (Capítol 3). La manca d'aquest control ha dificultat en algun moment el seguiment dels canvis fets al codi. Aquest aprenentatge, tot i haver arribat en forma de mancança més que d'èxit, es considera igualment valuós de cara a futurs projectes.

Capítol 12

Treball futur

Durant el desenvolupament de TerraLink s'han identificat diverses línies de millora que queden fora de l'abast d'aquest TFG, però que es plantegen com a properes passes naturals del projecte.

- **Integració de LoRa via mòdul USB:** tal com s'ha detallat al Capítol 2, la integració d'un mòdul LoRa directament als pins GPIO de la Raspberry Pi 5 no s'ha pogut completar. Es proposa substituir aquesta aproximació per un mòdul LoRa en format USB, que hauria d'evitar els problemes de compatibilitat detectats. Com que la Raspberry Pi 5 no disposa de prou ports USB lliures un cop connectats la resta de perifèrics, caldria adquirir també un *hub* USB alimentat per fer-ho possible.
- **Sistema de bateria/UPS:** el dispositiu actual no disposa de bateria pròpia (Capítol 2), cosa que el deixa sense funcionament davant un tall de subministrament elèctric, comproment fins i tot la disponibilitat del protocol d'emergència (RNF4, Capítol 10). Incorporar un sistema d'alimentació ininterrompuda (UPS) es considera una millora prioritària.
- **Targeta de so per a l'altaveu:** tal com s'ha detallat al Capítol 9, l'altaveu adquirit per al projecte no s'ha pogut connectar per manca d'una targeta de so de sortida. Incorporar-la permetria afegir un avís sonor en rebre missatges, millorant l'accessibilitat del sistema per a la Persona Dependent (RNF1).
- **Fallback automàtic WhatsApp→SMS per a missatges normals:** actualment, només el protocol d'emergència envia el missatge simultàniament per tots els canals disponibles; l'enviament d'un missatge de veu normal (RF1) només s'intenta per WhatsApp (Capítol 9). Es proposa estendre aquest comportament de manera que, si WhatsApp no està disponible, el sistema recorri automàticament a SMS també per als missatges normals, completant així el requisit RNF2 (Capítol 10).
- **Centralització de la configuració del sistema:** diversos paràmetres estan actualment definits com a constants dins el codi font: rutes de fitxers (`m_rutaFitxer` a `RepositoriContactes`), el port de la impressora (`m_port`), el directori de treball i l'script de WhatsApp (`AdaptadorWhatsApp`), l'adreça IP del mòdem Huawei (192.168.8.1, repetida a diversos punts d'`AdaptadorModem4G`) o el dispositiu ALSA del micròfon (`hw:3,0`). Es proposa centralitzar-los en un fitxer de configuració global o una classe de configuració dedicada, permetent modificar-los sense haver de recompilar l'aplicació.

- **Control de versions:** tal com s'ha reconegut al Capítol 3, el projecte no ha utilitzat cap sistema de control de versions (com Git) durant el seu desenvolupament. Incorporar-ne un des del primer dia de qualsevol ampliació futura permetria mantenir un historial de canvis i facilitar-ne el manteniment col·laboratiu.
- **Proves unitàries automatitzades:** les proves realitzades durant aquest projecte han estat manuals i basades en traves de `qDebug()` (Capítol 9). Es proposa incorporar un framework de testing automatitzat, com `QtTest`, per cobrir de manera sistemàtica les classes principals del sistema (`RepositoriContactes`, `MissatgeFactory`, `GestorEmergencia`, entre d'altres).
- **Actualitzacions automàtiques del programari:** de cara a un escenari de comercialització (Capítol 2), es planteja la incorporació d'un mecanisme d'actualització automàtica del programari, que hauria d'estudiar-se amb especial atenció als aspectes de seguretat que comporta.
- **Marcatge CE i normativa de protecció de consumidors:** en cas de comercialitzar el dispositiu, caldria revisar el compliment del marcatge CE i la normativa de protecció de consumidors aplicable, aspectes que queden fora de l'abast d'aquest TFG (Capítol 2).
- **Ampliació de la gestió de fotografies i missatges:** tal com s'ha comentat al Capítol 11, el sistema actual permet rebre i imprimir fotografies, però no ofereix una experiència de navegació o galeria de missatges. Es proposa ampliar la interfície per permetre consultar l'historial de fotografies i missatges rebuts de manera més rica i visual.

Apèndix A

Manuels d'instal·lació i d'usuari

A.1 Manual d'instal·lació (configuració de fàbrica)

A diferència d'un projecte de programari convencional, TerraLink es concep com un dispositiu que arriba a mans de l'usuari final **ja preconfigurat de fàbrica**: el propi sistema operatiu, l'aplicació compilada i el servei d'arrencada automàtica ja estan instal·lats i en funcionament abans que el Cuidador rebi el dispositiu. Aquesta secció documenta, doncs, el procés que segueix el fabricant (o l'estudiant, en aquesta fase de TFG) per preparar un dispositiu nou, no una tasca que hagi de repetir l'usuari final.

A.1.1 Requisits previs

- Raspberry Pi 5 amb Raspberry Pi OS instal·lat.
- Connexió a internet (WiFi o mòdem 4G) durant la instal·lació.
- Mòdem 4G Huawei E3372, impressora tèrmica ESC/POS, micròfon/altaveu USB i teclat Pico ja connectats per USB.
- Pantalla connectada via HDMI.

A.1.2 Instal·lació de dependències

Des d'un terminal a la Raspberry Pi, cal instal·lar les dependències següents:

```
sudo apt update
sudo apt install -y qt6-base-dev cmake ninja-build nodejs npm python3-pip
pip install faster-whisper --break-system-packages
```

A.1.3 Desplegament del codi

Copiar el projecte TerraLink al directori de treball i instal·lar les dependències de Node.js:

```
cd /home/paudomenech/TerraLink
npm install
```

Compilar l'aplicació:

```
cmake -B build -G Ninja
cmake --build build
```

A.1.4 Configuració de l'arrencada automàtica

Per fer que TerraLink s'engegui automàticament en arrencar la Raspberry Pi, es crea un servei systemd. Cal crear el fitxer `/etc/systemd/system/terralink.service` amb el contingut següent:

```
[Unit]
Description=Terralink Qt App
After=graphical.target
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=paudomenech
WorkingDirectory=/home/paudomenech/TerraLink
Environment=DISPLAY=:0
ExecStart=/home/paudomenech/TerraLink/build/TerraLink
#Restart=always
#RestartSec=2

[Install]
WantedBy=graphical.target
```

Un cop creat el fitxer, cal activar i engegar el servei:

```
sudo systemctl daemon-reload
sudo systemctl enable terralink.service
sudo systemctl start terralink.service
```

Per comprovar que el servei s'ha engegat correctament:

```
sudo systemctl status terralink.service
```

Un cop completada aquesta configuració de fàbrica, el dispositiu queda llest per ser lliurat a l'usuari final. L'única tasca que li quedarà pendent al Cuidador és la configuració inicial de contactes i la vinculació del compte de WhatsApp, descrites a la Secció [A.2.5](#).

A.2 Manual d'usuari

Aquest manual explica com utilitzar el dispositiu TerraLink un cop rebut, ja preconfigurat, tant per a la Persona Dependent que l'usa diàriament com per al Cuidador que se n'encarrega de la configuració inicial dels contactes.

A.2.1 Navegació bàsica

TerraLink es controla amb el teclat de 3 tecles connectat al dispositiu, senyalitzat amb icones intuïtives perquè la Persona Dependent no necessiti conèixer cap terme

tècnic:

- **Fletxa avall** (↓): mou la selecció cap a l'element següent de la pantalla.
- **Fletxa amunt** (↑): mou la selecció cap a l'element anterior de la pantalla.
- **Confirmar** (botó dret, marcat amb un cercle): confirma o activa l'element seleccionat.

L'element seleccionat en cada moment es marca visualment amb un requadre de color a la pantalla.

A.2.2 Enviar un missatge de veu

1. A la pantalla principal, seleccionar l'opció "Enviar Missatge" i prémer **Confirmar**.
2. Seleccionar el contacte al qual es vol enviar el missatge (canviar de contacte amb ↓/↑).
3. Prémer el botó de gravar i parlar amb normalitat.
4. Tornar a prémer el mateix botó per aturar la gravació.
5. Esperar que aparegui el text transcrit a la pantalla.
6. Seleccionar "Enviar" per confirmar l'enviament.

A.2.3 Rebre un missatge

Quan arriba un missatge nou, el sistema el mostra automàticament a la pantalla i, si la impressora tèrmica està activada, l'imprimeix en paper. Per consultar els missatges rebuts anteriorment, seleccionar l'opció "Missatges Rebuts" a la pantalla principal.

A.2.4 Activar una emergència

Des de la pantalla principal, seleccionar l'opció "Emergència" i prémer **Confirmar**. El sistema enviarà immediatament un avís a tots els contactes marcats com a contactes d'emergència, utilitzant tots els canals de comunicació disponibles simultàniament (Capítol 9).

A.2.5 Configuració (només per al Cuidador)

La pantalla de configuració no és accessible des del teclat de 3 tecles habitual de la Persona Dependent, precisament per evitar-hi accessos accidentals. Per accedir-hi, el Cuidador ha de:

1. Connectar un teclat estàndard (qualsevol teclat USB de casa) a la Raspberry Pi.
2. Prémer, en aquest ordre, les tecles **1, 2, 3, 4**.

Vincular el compte de WhatsApp

1. Dins la pantalla de configuració, seleccionar "QR WhatsApp".
2. El sistema mostrarà un codi QR en pantalla.
3. Obrir WhatsApp al mòbil que es vulgui vincular, anar a Configuració → Dispositius vinculats → Vincular un dispositiu, i escanejar el codi QR mostrat.
4. Un cop vinculat, la pantalla mostrarà la confirmació "WhatsApp connectat".

Gestionar contactes

1. Dins la pantalla de configuració, seleccionar "Contactes".
2. Per importar automàticament els contactes del compte de WhatsApp vinculat, seleccionar "Importar WA".
3. Per afegir un contacte manualment, seleccionar "Afegir" i introduir el nom i el número de telèfon.
4. Per eliminar o modificar un contacte, seleccionar-lo de la llista i triar l'opció corresponent.

Marcar contactes d'emergència

Actualment, marcar un contacte com a contacte d'emergència requereix editar directament el fitxer `contactes.json` del dispositiu (funcionalitat pendent d'incorporar a la interfície, tal com es detalla al Capítol 12). Es recomana que aquesta tasca la faci un tècnic o el Cuidador amb el suport de l'estudiant durant la fase de configuració inicial.

Apèndix B

Prova amb *vibe coding*: prototip descartat

Tal com s'ha comentat a la Secció 4.3, a l'inici de la fase d'implementació es va dur a terme una primera prova de codi generat de forma majoritàriament automàtica mitjançant eines d'intel·ligència artificial generativa, sense un disseny previ ni una revisió manual acurada del resultat: l'anomenat *vibe coding*. Aquest apèndix documenta l'abast d'aquesta prova, els problemes detectats i les raons per les quals es va descartar íntegrament del projecte final.

B.1 Objectiu de la prova

L'objectiu no era obtenir codi de producció, sinó validar ràpidament que era tècnicament possible comunicar-se amb cada component de maquinari (mòdem 4G, impressora tèrmica, micròfon, etc.) abans d'invertir temps en un disseny orientat a objectes complet. Es tractava, doncs, d'una prova de concepte exploratòria, feta en Python, demanant a una eina d'IA generativa que escrivís directament el codi necessari per a cada integració, amb molt poca supervisió del resultat.

B.2 Problemes detectats

La revisió d'aquest prototip va posar de manifest diversos problemes, comuns a molt codi generat sense disseny ni revisió prèvia:

- **Manca d'arquitectura:** el codi no seguia cap patró de disseny ni separació de responsabilitats. Tota la lògica (comunicació amb el maquinari, gestió d'errors, lògica de negoci) es trobava barrejada en scripts poc estructurats.
- **Duplicació i inconsistència:** funcions molt similars es repetien amb petites variacions en diferents fitxers, en lloc de reutilitzar-se, cosa que dificultava saber quina versió era la "correcta".
- **Gestió d'errors superficial:** la majoria de crides a components externs (mòdem, impressora) no contemplaven casos de fallada real, i sovint es limitaven a capturar excepcions genèriques sense cap acció correctiva.

- **Dificultat de manteniment:** en no haver participat en el disseny del codi, a l'estudiant li costava predir com es comportaria davant de canvis, ni raonar-hi amb seguretat, cosa que en feia inviable l'ampliació progressiva.
- **Baixa confiança en la correcció:** sense haver dissenyat ni escrit el codi, l'estudiant no podia garantir-ne la correcció més enllà de provar-lo manualment cas per cas, a diferència del codi final, escrit i raonat directament per l'estudiant.

Per aquests motius, es va descartar íntegrament aquest prototip com a base del projecte final. Sí que va resultar útil, però, per identificar ràpidament quines llibreries i crides eren necessàries per a cada component de maquinari, informació que es va aprofitar durant el disseny (Capítol 8) i la implementació manual i acurada del sistema final (Capítol 9).

B.3 Conclusió

Aquesta prova reforça una de les conclusions metodològiques d'aquest TFG (Capítol 11): la intel·ligència artificial generativa és una eina d'ajuda vàlida per a tasques puntuals (depuració, suggeriments de disseny, redacció), però delegar-hi directament la generació de l'arquitectura i el codi final d'un sistema, sense disseny ni revisió humana, produeix un resultat de baixa qualitat i poc mantenible, inadequat per a un projecte que ha de perdurar més enllà d'una prova ràpida.

B.4 Codi del prototip

El codi complet d'aquest prototip descartat, conservat únicament amb finalitat documental i de transparència metodològica, es pot consultar a:

Codi vibe-coding: <https://github.com/PauDomenech/tfg-terralink/tree/main/vibecode>

Bibliografia

- [1] Parlament Europeu i Consell de la Unió Europea. *Reglament (UE) 2016/679 del Parlament Europeu i del Consell, de 27 d'abril de 2016, relatiu a la protecció de les persones físiques pel que fa al tractament de dades personals i a la lliure circulació d'aquestes dades (RGPD)*. 2016. URL: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679> (cons. 11-06-2026).
- [2] Jefatura del Estado español. *Llei Orgànica 3/2018, de 5 de desembre, de Protecció de Dades Personals i garantia dels drets digitals*. 2018. URL: <https://www.boe.es/eli/es/lo/2018/12/05/3> (cons. 11-06-2026).
- [3] Jefatura del Estado español. *Ley 34/2002, de 11 de julio, de Servicios de la Sociedad de la Información y de Comercio Electrónico*. 2002. URL: <https://www.boe.es/buscar/act.php?id=B0E-A-2002-13758> (cons. 08-06-2026).
- [4] The Qt Company. *Qt Creator IDE*. 2024. URL: <https://www.qt.io/development/tools/qt-creator-ide> (cons. 25-06-2026).
- [5] Dimitri van Heesch. *Doxygen: Generate documentation from source code*. 2024. URL: <https://www.doxygen.nl/> (cons. 05-07-2026).
- [6] JGraph Ltd. *draw.io*. 2024. URL: <https://www.drawio.com/> (cons. 28-06-2026).
- [7] Meshtastic Project. *Meshtastic – Open Source Off-Grid Mesh Communication*. 2024. URL: <https://meshtastic.org/> (cons. 14-06-2026).
- [8] Meshtastic Project. *Primeros pasos – Documentación Meshtastic*. Guia introductòria en castellà sobre la configuració de dispositius Meshtastic. 2024. URL: <https://meshtastic.es/docs/primeros-pasos/> (cons. 14-06-2026).
- [9] LabelDictate. *Diseño de formatos de etiquetas y tickets para la impresión de archivos CSV y TXT con ESC/POS*. Consultat com a referència general sobre el funcionament del protocol ESC/POS. 2024. URL: <https://ayuda.labeldictate.com/2024/05/10/disenio-de-formatos-de-etiquetas-y-tickets-para-la-impresion-de-archivos-csv-y-txt-con-esc-pos/> (cons. 02-07-2026).
- [10] Huawei Technologies Co., Ltd. *Huawei E3372H-153 – Manual de usuario*. 2023. URL: <https://www.manualpdf.es/huawei/e3372h-153/manual> (cons. 20-06-2026).
- [11] Pedro S. Lopez i contributors. *webjs.dev – Documentación oficial de whatsapp-web.js*. 2024. URL: <https://webjs.dev/> (cons. 05-06-2026).
- [12] Guillaume Klein i contributors. *faster-whisper: Faster Whisper transcription with CTranslate2*. 2024. URL: <https://github.com/SYSTRAN/faster-whisper> (cons. 01-07-2026).
- [13] JLCPCB. *Arduino vs Raspberry Pi: Which One Should You Choose?* 2024. URL: <https://jlcpcb.com/blog/arduino-vs-raspberry-pi> (cons. 03-07-2026).
- [14] Russ Miles i Kim Hamilton. *Learning UML 2.0*. Sebastopol, CA: O'Reilly Media, 2006.

-
- [15] OpenJS Foundation. *Node.js Documentation*. 2024. URL: <https://nodejs.org/en/docs/> (cons. 17-06-2026).